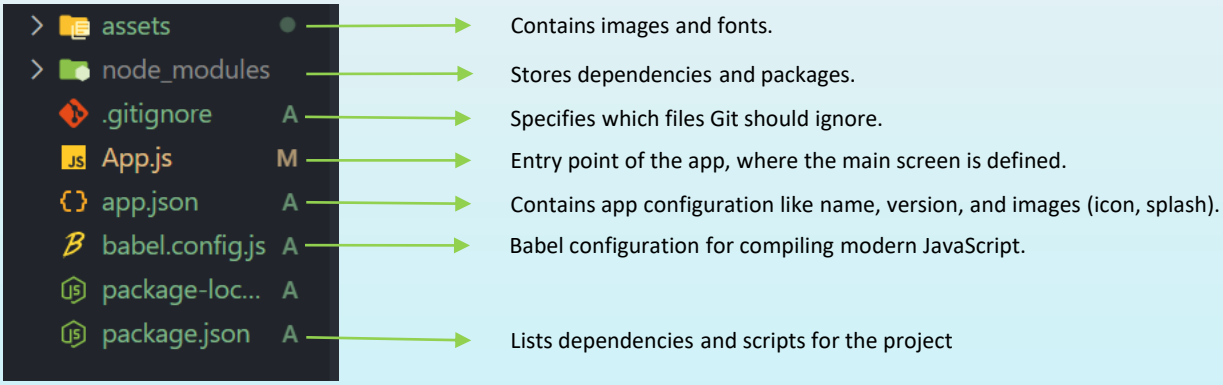
React Native

With Expo CLI

React Native : Notes Index

React-Native Create Project & Folder Structure

1 `npx create-expo-app@latest --template blank@latest`



2 App.jsx

```jsx
import { StatusBar } from "expo-status-bar";
import { StyleSheet, Text, View } from "react-native";

export default function App() {
  return (
    <View style={styles.container}>
      <Text>Hello World</Text>
    </View>
  );
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    backgroundColor: "#fff",
    alignItems: "center",
    justifyContent: "center",
  },
});
```

3 `npx expo start`

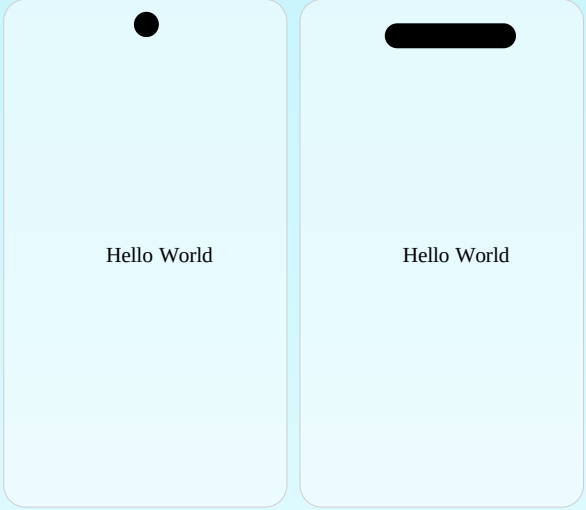
Note:

Start the Development Server:
It launches the Expo development server.

Opens Metro Bundler:
A web interface opens in your browser showing a QR code and various options for debugging.

Scan QR Code: Use the Expo Go app on your mobile device to scan the QR code and preview your app in real-time.

Live Reload: Any changes you make to your code are automatically reloaded on your device without needing to restart the app.



1 App.js — Example of : View component

```
import React from "react";
import { View, Text } from "react-native"; //import View,Text

export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: "center",alignItems: "center" }}>//view component
      <Text>Hello World</Text>
    </View>
  );
}
```

2 App.js — Example of : Text component

```
import React from "react";
import { View, Text } from "react-native"; //import View, Text

export default function App() {
  return (
    <View style={{flex: 1, padding: 20, alignItems: "center", justifyContent: "center",}}>
      <Text style={{ fontWeight: "bold", fontSize: 20 }}>Bold Text</Text> //text component
    </View>
  );
}
```

3 App.js — Example of : Image component

```
import React from "react";
import { View, Image } from "react-native";
import abhiyanshImage from "./assets/abhiyansh.jpg"; //import image from asset folder

export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Image
        source={{ uri: "https://picsum.photos/200" }} //display image from URL
        style={{ width: 200, height: 200 }} />

      <Image source={abhiyanshImage} /> //display image from asset folder
    </View>
  );
}
```

Basic React Native components
React Native provides several core components to build the UI of your app. Each component is a building block of the interface. Let's go through some of the essential ones and see how to use them

View
The View component is like a div in HTML. It is a container that can hold other components and define layout, styling, etc.

Text
The Text component is used to display text in the app.

Image
The Image component is used to display images from a URL or a local resource.

Hello World

Bold Text

Hello World

Bold Text

1 App.js Example of : Button component

```
import React from "react";
import { View, Button, Alert } from "react-native"; //import View, Button & Alert

export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Button title="Press me" onPress={() => Alert.alert("Button pressed!")} />
    </View>
  );
} //when user press button onPress event tiger and alert box open
```

2 App.js Example of : TextInput component

```
import React, { useState } from "react"; //input useSatate
import { View, Text, TextInput } from "react-native"; //import TextInput

export default function App() {

  const [input, setInput] = useState(""); //initialize useState to hold & update input value

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>

      <TextInput
        style={{
          height: 40,
          width: 200,
          borderColor: "gray",
          borderWidth: 1,
          padding: 5,
        }} //style the Text input

        placeholder="Type here" //place holder for Text Input
        onChangeText={(text) => setInput(text)} //update input value on : onChangeText event
        value={input} //set Text Input value
      />
      <Text>You type : {input}</Text> //show input value in Text component
    </View>
  );
}
```

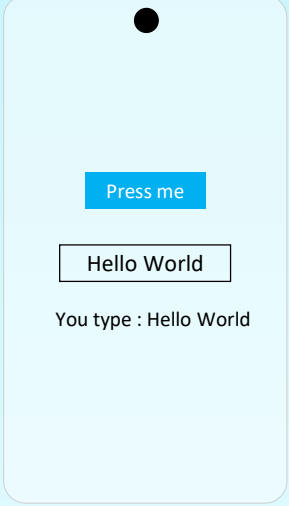
Button
The Button component renders a platform-specific button that users can press.

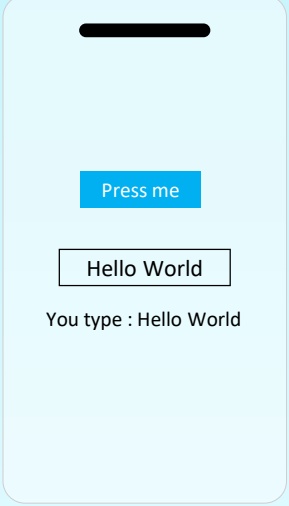
TextInput
The TextInput component allows the user to input text.

State Management:
You're using the useState hook to manage the input state.

TextInput: The TextInput component has an onChangeText handler that updates the input state whenever the user types.

Displaying Input: The current value of input is displayed in a Text component below the input field.





React-Native Pressable, TouchbaleOpacity

1 App.js — Example of : Pressable component

```
import React from "react";
import { View, Text, Pressable } from "react-native"; //import component



export default function App() {

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
-------------------------------------------------------------------------------------------

      <Pressable            //define pressable component for click me text
        style={({ pressed }) => [
          { backgroundColor: pressed ? "#d3d3d3" : "red" },
          { padding: 10 },
        ]}
        onPress={() => alert("Press me")}
      >
        <Text>Click Me</Text>
      </Pressable>

-------------------------------------------------------------------------------------------

    </View>
  );
}
```

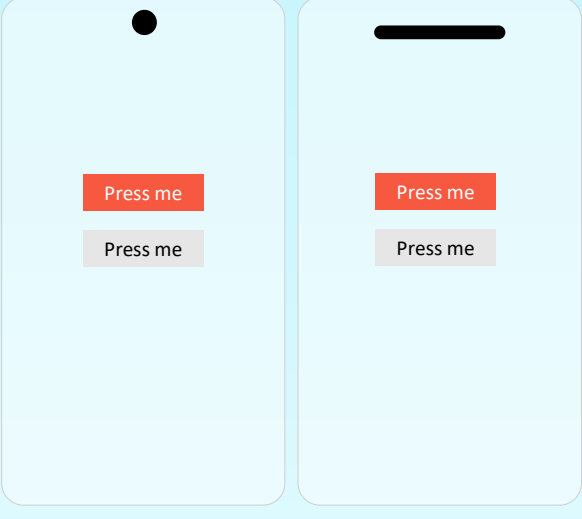
2 App.js — Example of : TouchbaleOpacity component

```
import React from "react";
import { View, Text, TouchableOpacity } from "react-native"; //import component


export default function App() {
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <TouchableOpacity onPress={() => alert("T Press me ")}>
        <Text>Click Me</Text>
      </TouchableOpacity>
    </View>
  );
}
```

Pressable and TouchableOpacity Explanation

PressableDefinition: A modern component that allows you to detect various user interactions, like presses, long presses, hover, and focus.Advantages:Highly customizable.Supports additional events like onPressIn, onPressOut, and onHoverIn.

TouchableOpacityDefinition: A simpler component that changes the opacity of the button when pressed, providing visual feedback.Use Case: Ideal for straightforward interactions, like buttons or links, where you want simple visual feedback.



React-Native SafeAreaView, Dimensions, Pixel Ratio

1 App.js — Example of : SafeAreaView

```
import React from "react";
import { Text, Button } from "react-native";
import { SafeAreaView } from "react-native-safe-area-context"; //import SafeAreaView

export default function App() {
  return (
    <SafeAreaView //Wrap all component in SafeAreaView Container
      style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>

      <Text>Hello, SafeAreaView!</Text>
      <Button title="Click Me" onPress={() => alert("Button Pressed")} />
    </SafeAreaView>
  );
}
```

1 App.js — Example of : Dimensions & Pixel Ratio

```
import React from "react";
import { View, Text, StyleSheet, Dimensions, PixelRatio } from "react-native";

const { width, height } = Dimensions.get("window"); // Get screen dimensions
const fontSize = PixelRatio.getFontScale() * 14; // Calculate font size based on screen pixel
density

const App = () => {
  return (
    <View style={styles.container}>
      <Text style={{ fontSize }}>Responsive Font Size</Text>
      <Text>Screen Width: {Math.round(width)} px</Text>
      <Text>Screen Height: {Math.round(height)} px</Text>
      <View style={[styles.box, { width: width * 0.6, height: height * 0.1 }]}>
        <Text style={styles.boxText}>Responsive Box</Text>
      </View>
    </View>
  );
};

const styles = StyleSheet.create({

  container: {flex: 1, justifyContent: "center", alignItems: "center",},
  box: { backgroundColor: "skyblue", justifyContent: "center", alignItems: "center",},
  boxText: { color: "white",},
});
export default App;
```

SafeAreaView
from react-native-safe-area-context ensures the UI respects system-safe areas like the notch or status bar.
justifyContent: "center" and alignItems: "center" safely center the Text and Button within the available screen space.flex: 1 makes the SafeAreaView cover the entire screen height.

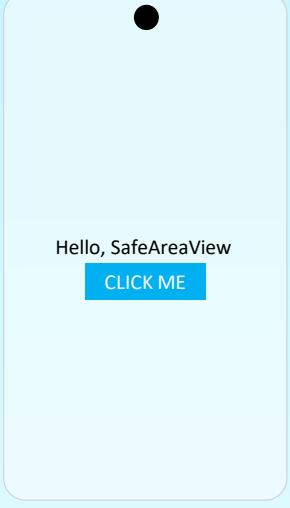
Dimensions API:
Dimensions.get('window') retrieves the device's screen width and height.

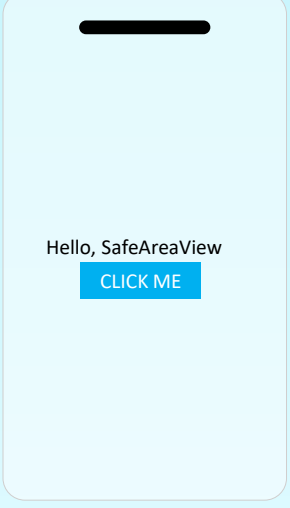
Usage in Styling:
width: width * 0.8: Makes the box's width 80% of the screen width.height: height * 0.2: Sets the height to 20% of the screen height.

1 App.js Example of : KeyboardAvoidingView

```
import React, { useState, useEffect } from "react";
import { View, TextInput, Button, KeyboardAvoidingView, Platform, TouchableWithoutFeedback,
  Keyboard, } from "react-native";

export default function App() {
  const [inputValue, setInputValue] = useState("");
  const [keyboardVisible, setKeyboardVisible] = useState(false);

  useEffect(() => {
    const showListener = Keyboard.addListener("keyboardDidShow", () =>
      setKeyboardVisible(true)
    );
    const hideListener = Keyboard.addListener("keyboardDidHide", () =>
      setKeyboardVisible(false)
    );
    return () => {
      showListener.remove();
      hideListener.remove();
    };
  }, []);

  return (
    <KeyboardAvoidingView
      style={{ flex: 1 }}
      behavior={Platform.OS === "ios" ? "padding" : "height"}
    >
      <TouchableWithoutFeedback onPress={Keyboard.dismiss}>
        <View
          style={{ flex: 1, justifyContent: "center", alignItems: "center" }}
        >
          <TextInput
            style={{ height: 40, borderColor: "gray", borderWidth: 1, marginBottom: 20,
              width: "80%",}}
            placeholder="Type here"
            value={inputValue}
            onChangeText={setInputValue}
          />
          {!keyboardVisible && (
            <Button title="Submit" onPress={() => alert("Form Submitted!")} />
          )}
        </View>
      </TouchableWithoutFeedback>
    </KeyboardAvoidingView>
  );
}
```

This React Native app manages keyboard visibility by using keyboardDidShow and keyboardDidHide event listeners. When the keyboard is visible, the layout adjusts using KeyboardAvoidingView. Tapping outside the input field dismisses the keyboard with TouchableWithoutFeedback. The submit button only appears when the keyboard is hidden.





1 App.js Example of : ScrollView component

```
import React from "react";
import { View, Text, ScrollView } from "react-native"; //import ScrollView

export default function App() {
  return (
    <ScrollView> //scroll view for long data rendering
      <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
        <Text>Scroll through the content below:</Text>

        {Array.from({ length: 50 }).map((_, i) => (//using map method to generate long data
          <Text key={i} style={{ margin: 10 }}>
            Item {i + 1}
          </Text>
        ))}

      </View>
    </ScrollView>
  );
}
```

1 App.js Example of : FlastList component

```
import React from "react";
import { View, Text, FlatList } from "react-native"; //import FlatList

const data = [ { id: "1", name: "John" }, { id: "2", name: "Jane" },{ id: "3", name: "Paul"
},]; //data object

export default function App() {
  return (
    <View>
      <FlatList //flat list component using for long data optimize performance
        data={data} //pass data you wont show in flat list
        keyExtractor={(item) => item.id} //this props extract key from data

        renderItem={({ item }) => (//renderItem props define all rendering logic for FlatList
          <Text style={{ padding: 20 }}>{item.name}</Text> //showing data.name in FlatList
        )}
      />
    </View>
  );//by default FlatList is Vertical scrolling pass horizontal props for horizontal scrolling
}
```

ScrollView
The ScrollView component allows for scrolling when the content is too long to fit on the screen

FlatList
FlatList is used for rendering a list of data efficiently, especially for large datasets

Props list
data: The array of items to display in the list.
renderItem: Function to render each item.
keyExtractor: Function to provide unique keys for each item.
onEndReached:
Callback triggered when the list reaches the end loading more data).
onEndReachedThreshold: How close to the end before onEndReached is trigger
refreshControl: Used for pull-to-refresh functionality.
ListEmptycomponent: component to show when the list is empty.
initialNumToRender: Number of items to render initially (improves performance).
horizontal: If true, the list scrolls horizontally

Item1
Item2
Item3
Item4
Item5
Item6
Item7

Item1
Item2
Item3
Item4
Item5
Item6
Item7

1 App.js Example of : SectionList component

```
import React from "react";
import { View, Text, SectionList } from "react-native"; //import SectionList

const DATA = [ //fake object data of fruits & vegetables for testing
  {
    title: "Fruits",
    data: ["Apple", "Banana", "Orange"],
  },
  {
    title: "Vegetables",
    data: ["Carrot", "Broccoli", "Spinach"],
  },
];

export default function App() {
  return (

<View style={{flex: 1, justifyContent: "center", alignItems: "center", padding: 20,}}>

<SectionList

//The array of data for each section.
sections={DATA}

//Function to provide unique keys for list items.
keyExtractor={(item, index) => item + index}

//Function to render each item in a section
renderItem={({ item }) => <Text>{item}</Text>}

 //Function to render the header for each section.
 renderSectionHeader={({ section: { title } }) => (

          <View>
              <Text style={{ fontWeight: "bold" }}>{title}</Text>
          </View>
  )}/>

   </View>
  );
}
```

SectionList
SectionList is used for rendering a list with sections, making it useful when you need to group data (e.g., displaying items categorized under different headings)

Key Features:
Data: Like FlatList, it takes an array of data, but each section can have its own data array.

Sections: Each section contains a title and data property. The title is displayed as the section header, and data contains the items for that section.

renderSectionHeader: Renders the header for each section.

renderItem: Renders each item within a section.

Key Props (Essential):
sections: The array of data for each section.renderItem: Function to render each item in a section.renderSectionHeader: Function to render the header for each section.keyExtractor: Function to provide unique keys for list items.

.

Fruits
Apple
Orange

Vegetables
Carrot
Broccoli
Spinach

Fruits
Apple
Orange

Vegetables
Carrot
Broccoli
Spinach

1 App.js Example of : Styling

```
import React from "react";
import { View, Text, StyleSheet, Dimensions } from "react-native";

// Get screen dimensions
const { width, height } = Dimensions.get("window");

// Define styles outside the component function
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "#f0f0f0",
  },
  title: { fontSize: 24, fontWeight: "bold", color: "#333",},
  subtitle: { fontSize: 18, fontWeight: "bold", color: "#666",},

  box: {
    width: "50%", // 50% of the parent width
    backgroundColor: "#e0e0e0",
    padding: 10, alignItems: "center",  marginBottom: 10,
  },
  responsiveBox: {
    width: width * 0.7, // 70% of screen width
    backgroundColor: "#d0d0d0", padding: 10, alignItems: "center",
  },
});

export default function App() {
  return (
    <View style={styles.container}> {/* apply container style */} of styles object
      <Text style={styles.title}>Hello, World!</Text> //apply title style
      <Text style={styles.subtitle}>Welcome to React Native Styling</Text>
      <View style={styles.box}>
        <Text>This is 50% width of the parent</Text>
      </View>
      <View style={styles.responsiveBox}>
        <Text>This box adjusts to 70% of the screen width!</Text>
      </View>
    </View>
  );
}
```

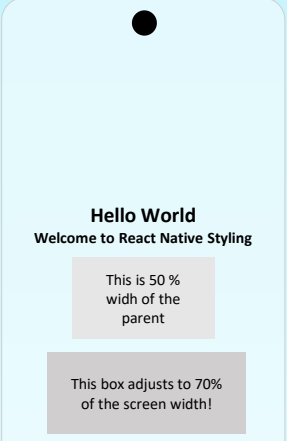
Explanation:

Dimensions API:
The Dimensions.get('window') retrieves the width and height of the device screen.

Responsive Styling:
The box style is set to 50% of the parent container's width.
The responsiveBox style uses the width variable from the Dimensions API to set its width to 70% of the screen width, making it responsive to different screen sizes.

This example demonstrates how you can use both inline styles and the Dimensions API to create responsive designs in React Native

Common Styling Properties:
Colors: You can use named colors, hex codes, or RGB values.
Font Size: fontSize: 16 sets the text size.
Margins and Padding: Control spacing with margin and padding properties.
Borders: Use borderWidth, borderColor, and borderRadius to style borders..



Stack Navigation Basic Stack Navigation

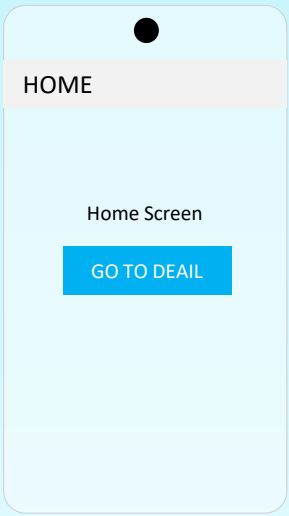
1 **npx expo install @react-navigation/native @react-navigation/native-stack react-native-gesture-handler react-native-reanimated react-native-screens react-native-safe-area-context**

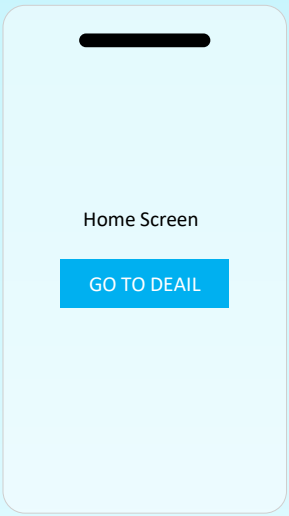
2 App.js

```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { Text, View, Button } from "react-native"; //import NavigationContainer & others

const Stack = createNativeStackNavigator(); // Create the stack navigator
-----------------------------------------------------------------------------------------

function HomeScreen({ navigation }) { //home screen component pass navigation props
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Home Screen</Text>
      <Button
        title="Go to Details"
        onPress={() => navigation.navigate("Details")} //button for navigate detail screen
      />
    </View>
  );
}
-----------------------------------------------------------------------------------------

function DetailsScreen({ navigation }) { //Detail screen component pass navigation props
  return (
    <View>
      <Text>Details Screen</Text>
      <Button title="Go to Home" onPress={() => navigation.navigate("Home")} />
    </View> //button to navigate home screen on onPress event
  );
}
-----------------------------------------------------------------------------------------

export default function App() {
  return (
    <NavigationContainer> //all Stactk Navigation in side NavigationContainer component
      <Stack.Navigator initialRouteName="Home"> //initialRouteName props for index screen
        <Stack.Screen name="Home" component={HomeScreen} /> //route for home screen
        <Stack.Screen name="Details" component={DetailsScreen} /> //route for detail screen
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Basic Stack Navigation
The **Stack Navigator** allows you to manage screens in a way that feels like a stack of pages. Users can push and pop screens, just like browsing through a website.

Key points:
NavigationContainer: The main container that wraps your navigators.
Stack.Navigator: Defines the type of navigator (here it's a stack).
initialRouteName: Sets the initial screen.
Stack.Screen: Defines the individual screens in your navigator.





Stack Navigation Navigator Option list

Property/Key	Short Description
initialRouteName	Specifies the first screen to render in the stack. Default is the first screen defined.
screenOptions	Default options for all screens in the stack navigator.
headerShown	Toggles the visibility of the header. Default: true.
headerTitle	Customizes the title text displayed in the header.
headerStyle	Styles the container of the header (e.g., background color).
headerTintColor	Sets the color of the header’s title and back button.
headerTitleStyle	Styles the text of the header title.
headerBackTitleVisible	Toggles visibility of the back button’s text. Default: true.
animation	Determines the animation style between screens. Values: "default", "fade", "none", etc.
gestureEnabled	Enables or disables gestures for navigation. Default: true.
headerShadowVisible	Shows or hides the shadow under the header. Default: true.
detachPreviousScreen	Unmounts the previous screen when it is no longer visible. Default: false.
presentation	Determines how the screen is presented. Values: "card", "modal", "transparentModal".
fullScreenGestureEnabled	Allows gestures to work across the entire screen. Default: false.
statusBarStyle	Sets the style of the status bar. Values: "auto", "inverted", "light", "dark".
statusBarAnimation	Animation style for status bar transitions. Values: "none", "fade", "slide".
lazy	Delays rendering inactive screens. Default: true.
unmountOnBlur	Unmounts screens when they lose focus. Default: false.
headerTransparent	Makes the header background transparent. Default: false.

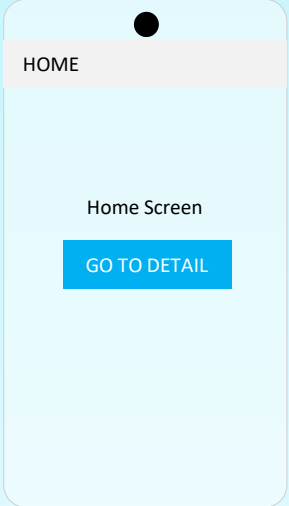
Stack Navigation-II Passing Data

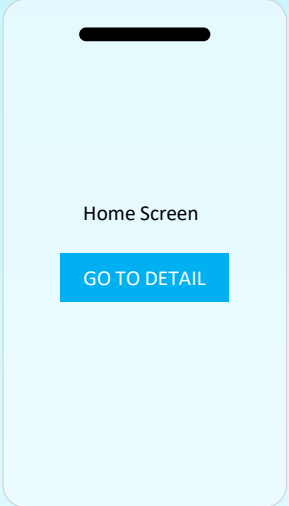
1 App.js

```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { Text, View, Button } from "react-native";


const Stack = createNativeStackNavigator(); // Create the stack navigator

-------------------------------------------------------------------------------------------

function HomeScreen({ navigation }) {//home screen component pass navigation props
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Home Screen</Text>
      <Button
        title="Go to Details"
        onPress={() => navigation.navigate("Details", { message: "Hello from" })}
      />               navigation.navigate(route name , data)
    </View>
  );
} //using Button to navigate Details screen and pass data in second argument

-------------------------------------------------------------------------------------------

function DetailsScreen({ route }) {  //data receive in route props
  const { message } = route.params; // data in params key
  return (
    <View>
      <Text>{message}</Text> //display message data in text component
    </View>
  );
}

-------------------------------------------------------------------------------------------

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
```

Explanation:
In HomeScreen, we navigate to the DetailsScreen using navigation.navigate('Details', { message: 'Hello from Home!' }).
Here, { message: 'Hello from Home!' } is the parameter being passed.

In DetailsScreen, we access the parameter via route.params.
The message is then displayed using {message}.





Stack Navigation-III Returning Data from the Second Screen

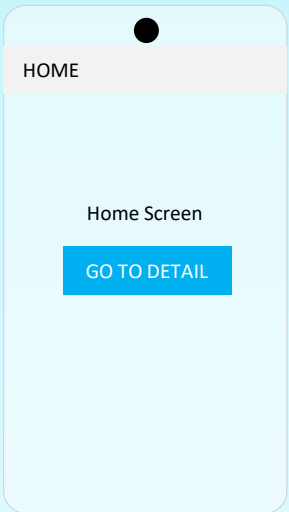
1 App.js

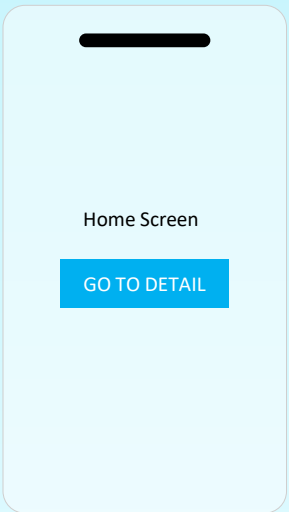
```
import React, { useEffect, useState } from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { View, Text, Button } from "react-native";

const Stack = createNativeStackNavigator(); //initialize navigator
-------------------------------------------------------------------------------------

function HomeScreen({ navigation, route }) { //getting data in route props
  const [Data, setData] = useState(null);

  useEffect(() => {
    if (route.params?.Data) {              //if data exist then set data
      setData(route.params?.Data);
    }
  }, [route.params?.Data]);                //only run when dependency array value change

  return (
    <View>
      <Text>Home Screen</Text>  <Text>Received : {Data}</Text>
      <Button title="Go to Select" onPress={() => navigation.navigate("SelectScreen")} />
    </View>
  );
}
-------------------------------------------------------------------------------------
function SelectScreen({ navigation }) {
return (
 <View>
   <Text>Select Screen</Text>
   <Button title="Return Data"onPress={() => navigation.navigate("Home", { Data: "Home" })}/>
 </View> //pass data in navigate(route,data) method as second argument
  );
}
-------------------------------------------------------------------------------------
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="SelectScreen" component={SelectScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

When navigating between screens, it's often useful to return data from the second screen back to the first. Here's how to handle this:In this example, we'll navigate to a second screen, pick a value there, and return it to the first screen.

Key Concepts:
Passing parameters back:
We use navigation.navigate to send data back to the previous screen.

React Hook (useEffect):
It listens to the route.params to update the state when the data is returned.





Stack Navigation-IV Style Header

1 App.js

```
import React, { useEffect, useState } from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { View, Text, Button, Alert } from "react-native"; //import all required dependencies

const Stack = createNativeStackNavigator();
-------------------------------------------------------------------------------------------

function HomeScreen({ navigation }) {      //home screen component pass navigation props
  return (
    <View>
      <Text>Home Screen With custom Header</Text>
      <Button title="Go to Detail“ onPress={() => navigation.navigate("Detail")} />
    </View>
  );
}
-------------------------------------------------------------------------------------------
function DetailScreen() {return (<View><Text>Details Screen</Text></View>);}
-------------------------------------------------------------------------------------------
const headerOption = {                                  //header option objecet
  title: "My custom home title",                        //header title
  headerStyle: { backgroundColor: "black", height: 400 }, //header style (css)
  headerTintColor: "red",                               //header title color
  headerRight: () => (                                  //place button on right
    <Button
      onPress={() => Alert.alert("Header Button Pressed!")} //onPress show alert
      title="Info"
      color="red" //button color
    />
  ),
};
-------------------------------------------------------------------------------------------
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home"> //initial home or route
        <Stack.Screen name="Home" component={HomeScreen} //rote for home screen
          options={headerOption} //pass header style and option in options props
        />
        <Stack.Screen name="Detail" component={DetailScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

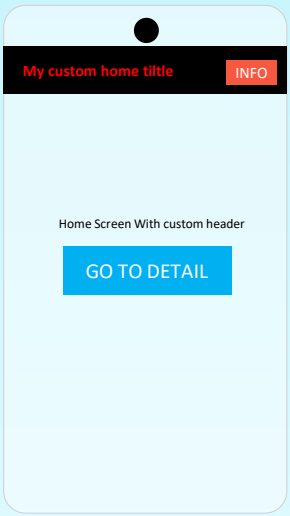
You can customize the header of your screens with options in the Stack Navigator. Here’s an example to change the header title, style, and add a custom button:Custom Header Example:

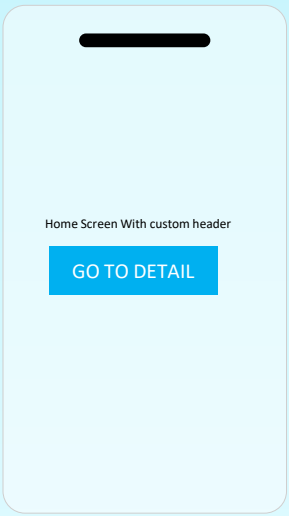
Key Concepts:

headerStyle:
Controls the background of the header.

headerTintColor:
Sets the color for the header text and icons.

headerRight:
Adds a custom button to the right side of the header.





Stack Navigation-V Custom Header

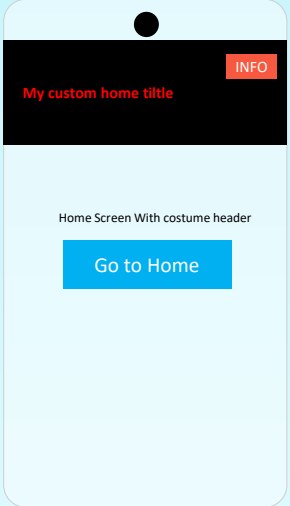
1 App.js

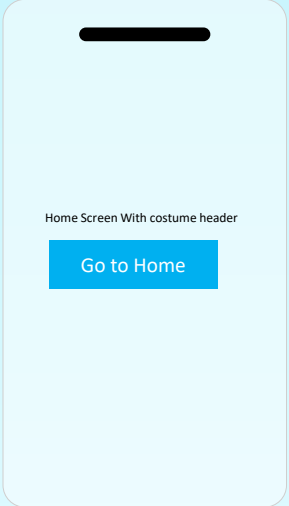
```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { View, Text, Button } from "react-native";
import CustomHeader from "./header/CustomHeader";

const Stack = createNativeStackNavigator(); // create stack navigator instance
-----------------------------------------------------------------------------------------------
function HomeScreen({ navigation }) {       //home screen component
  return (
    <View>
      <Text>Home Screen</Text>
      <Button
        title="go to Custom Header Screen"
        onPress={() => navigation.navigate("CustomHeader")} //navigate to customr Header
      />
    </View>
  );
}
-----------------------------------------------------------------------------------------------
function CustomHeaderScreen({ navigation }) {             //custom header screen component
  return (
    <View style={{ flex: 1 }}> //custom custom header component
      <CustomHeader title="Custom Header“ onPressBack={() => navigation.navigate("Home")}/>
      <View>
        <Text>Custom Header Screen</Text>
        <Button title="Go to Home" onPress={() => navigation.navigate("Home")} />
      </View>
    </View>
  );
}
-----------------------------------------------------------------------------------------------
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen
          name="CustomHeader"
          component={CustomHeaderScreen} //show custom navigation
          options={{ headerShown: false }}/> //hide default header of stack navigation
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Create the Custom Header component
First, let's create a simple header component. You can customize it with buttons, logos, and other elements.

Use the Custom Header component in the Screen
Next, import this CustomHeader component into your main App.js file and pass it to the options of the Stack Navigator.

Key Concepts
header in options: The header property allows you to pass a custom component. It takes a function that returns a React element, in this case, our custom header.navigation.goBack: This is used to handle the back button in the custom header, allowing the user to go back to the previous screen.Custom styling: The StyleSheet in the CustomHeader file allows for customized styling, such as colors, font size, alignment, etc.





Stack Navigation-VI Custom Header component-II

1 component\CustomHeader.js

```
import { View, Text, TouchableOpacity, StyleSheet, Button } from "react-native";

function CustomHeader({ title, onPressBack }) {
  return (
    <View style={styles.header}>
      <Button title="Back" onPress={onPressBack} />
      <Text style={styles.title}>{title}</Text>
    </View>
  );
}
const styles = StyleSheet.create({
  header: {
    height: 160,
    alignItems: "center",
    justifyContent: "center",
    padding: 10,
    backgroundColor: "black",
    elevation: 10,
  },
  title: {
    fontSize: 20,
    marginLeft: 10,
    color: "white",
  },
});

export default CustomHeader;
```

2nd way to use custom header

```
export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator initialRouteName="Home">
        <Stack.Screen
          name="Home"
          component={HomeScreen}
          options={{
          header: ({ navigation }) => (
               // Use custom header component
             <CustomHeader title="Home" navigation={navigation} />
            ),
          }}
        />
        <Stack.Screen
          name="Details"
          component={DetailsScreen}
          options={{
            header: ({ navigation }) => (
              <CustomHeader title="Details" navigation={navigation} />
            ),
          }}
        />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Stack Navigation-VII Customizing Transitions Between Screens

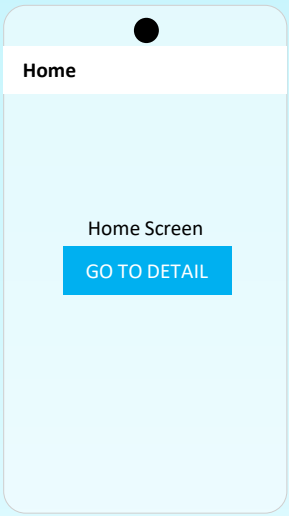
1 App.js

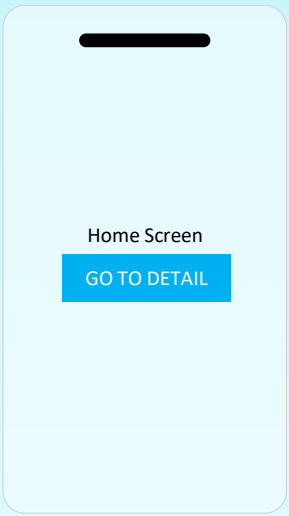
```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createNativeStackNavigator } from "@react-navigation/native-stack";
import { View, Text, Button } from "react-native";

const Stack = createNativeStackNavigator();
-----------------------------------------------------------------------------------------------

function HomeScreen({ navigation }) {
  return (
    <View>
      <Text>Home Screen</Text>
      <Button
        title="Go to Details"
        onPress={() => navigation.navigate("Details")}
      />
    </View>
  );
}
-----------------------------------------------------------------------------------------------

function DetailsScreen() {
  return (
    <View>
      <Text>Details Screen</Text>
    </View>
  );
}
-----------------------------------------------------------------------------------------------

export default function App() {
  return (
    <NavigationContainer>
      <Stack.Navigator
        initialRouteName="Home"
        screenOptions={{
          animation: "flip", // Custom transition animation
        }}
      >
        <Stack.Screen name="Home" component={HomeScreen} />
        <Stack.Screen name="Details" component={DetailsScreen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Available Screen Transition Animations

default: The default animation, which varies based on the platform
fade: Fades the new screen in and out.
slide_from_right: Slides the new screen in from the right (common for
slide_from_left: Slides the new screen in from the
left.slide_from_top: Slides the new screen in from the
top.slide_from_bottom: Slides the new screen in from the bottom.
flip: Flips the new screen into view.
fade_from_bottom: Fades the new screen in from the bottom

You can specify the desired animation type in the screenOptions of your stack navigator. Just replace the animation property with your chosen option.





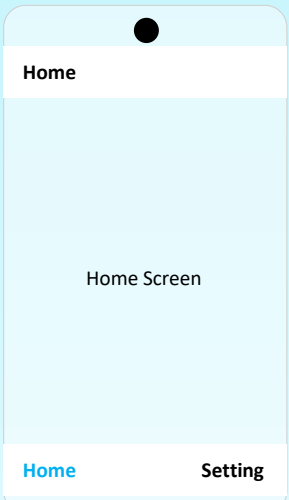
1 expo install @react-navigation/native @react-navigation/bottom-tabs

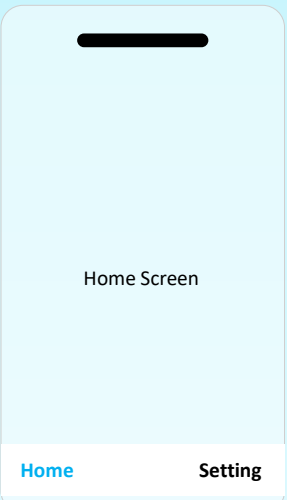
2 App.js

```
import React from "react";
import { View, Text } from "react-native";
import { NavigationContainer } from "@react-navigation/native";
import { createBottomTabNavigator } from "@react-navigation/bottom-tabs"; //import all

const Tab = createBottomTabNavigator(); //make instance of Tab Navigator
-----------------------------------------------------------------------------------------

function HomeScreen() {                    //home screen component
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Home Screen</Text>
    </View>
  );
}

-----------------------------------------------------------------------------------------

function SettingsScreen() {                //setting screen component
  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Text>Setting Screen</Text>
    </View>
  );
}

-----------------------------------------------------------------------------------------

export default function App() {
  return (
    <NavigationContainer> //navigation container
      <Tab.Navigator>
        <Tab.Screen name="home" component={HomeScreen} /> //route for home screen
        <Tab.Screen name="setting" component={SettingsScreen} /> //route for setting screen
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

Step 1: Install the Required Libraries

Tab Navigator: createBottomTabNavigator is used to create a tabbed navigation interface.
Screen components: HomeScreen and SettingsScreen are simple components displaying text. You can replace them with more complex UIs as needed.
NavigationContainer: This is a necessary wrapper around your navigator that manages the navigation state.
screenOptions: A function that determines how the tab icons are displayed based on the current route.
Tab.Screen: Each screen in the tab navigator is defined here, linking the screen name to its corresponding component
Imports: Necessary libraries are imported to build the navigation and UI.





Tab Navigator options list

Property/Key	Short Description
tabBarIcon	Defines the icon displayed in the tab bar for the screen.
tabBarLabel	Sets a custom label text for the tab.
tabBarBadge	Adds a badge (e.g., notification count) to the tab icon.
tabBarBadgeStyle	Customizes the style of the badge.
tabBarVisible	Controls the visibility of the tab bar (deprecated).
tabBarStyle	Customizes the style of the tab bar.
tabBarActiveTintColor	Sets the color of the icon and label when the tab is active.
tabBarInactiveTintColor	Sets the color of the icon and label when the tab is inactive.
tabBarButton	Provides a custom component to replace the default tab button.
tabBarHideOnKeyboard	Hides the tab bar when the keyboard appears.
tabBarLabelPosition	Positions the label relative to the icon ("below-icon", "beside-icon").
tabBarAllowFontScaling	Enables or disables font scaling for the tab label.
tabBarItemStyle	Customizes the style of individual tab items.
tabBarLabelStyle	Customizes the style of the label text.
tabBarIndicator	Adds a custom component for the tab indicator.
unmountOnBlur	Unmounts the screen when it loses focus.
headerShown	Controls the visibility of the header for the tab.
headerStyle	Customizes the style of the header.
headerTintColor	Sets the color of the header text and icons.
headerTitleAlign	Aligns the header title (left, center, etc.).
headerTitleStyle	Customizes the style of the header title.
lazy	Loads screens lazily when focused (performance optimization).
swipeEnabled	Enables swipe gestures between tabs.
keyboardHidesTabBar	Hides the tab bar when the keyboard appears (for iOS).
safeAreaInsets	Adjusts tab bar insets for safe areas on different devices.

Tab Navigation-II Customized Tab Bar

1 expo install @expo/vector-icons

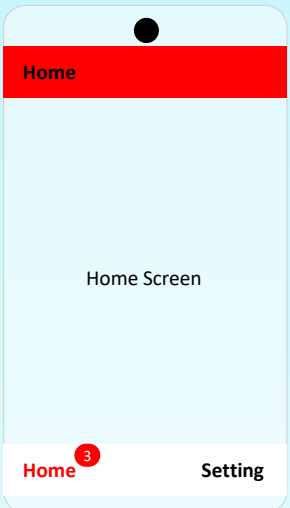
2 App.js

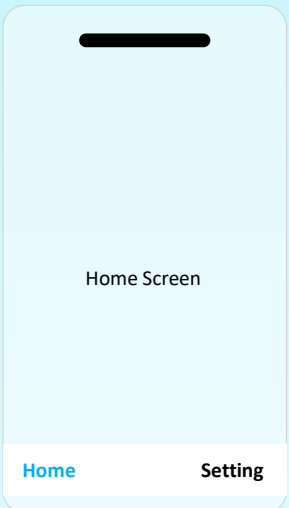
```
export default function App() {
  return (
    <NavigationContainer> //Wrap the navigator in NavigationContainer
      <Tab.Navigator
        screenOptions={({ route }) => ({// Customize Hearder
          headerStyle: {
            backgroundColor: "#f4511e",
          },
           //Set screen options based on the route, Define how the icons should look
            tabBarIcon: ({ focused, color, size }) => {
            let iconName;

            // Determine the icon name based on the route name
            if (route.name === "Home") {
              iconName = focused ? "home" : "home-outline"; // Home icon based on focus stat
            } else if (route.name === "Setting") {
              iconName = focused ? "settings" : "settings-outline"; // Settings icon
            }
             //return icon
            return <Ionicons name={iconName} size={size} color={color} />;
          },
          tabBarActiveTintColor: "tomato", //tab bar icon color when tab active
          tabBarInactiveTintColor: "gray", //tab bar icon color when tab inactive
          tabBarStyle: {
            backgroundColor: "#f2f2f2", //tab bar style
            height: 60,
          },
          tabBarLabelStyle: { //tab bar icon label style
            fontSize: 14,
            fontWeight: "bold",
           },
          tabBarShowLabel: true, // Show or hide tab labels
          tabBarHideOnKeyboard: true, //hide tab bar when keyboard is open
         lazy: true, // Loads tabs lazily (only when user navigates to them)
        })}>
        <Tab.Screen name="Home" component={HomeScreen} options={{
            tabBarBadge: 300, // Show a badge on the Home tab
          }}/>
        <Tab.Screen name="Setting" component={SettingsScreen} />
      </Tab.Navigator>
    </NavigationContainer>
  );
}
```

Key Customization Options:

tabBarIcon: A property to render the icons for the tab bar. The icon changes based on whether the tab is focused or not.

tabBarActiveTintColor:
Changes the color of the icon and label for the active tab.
tabBarInactiveTintColor:
Changes the color of the icon and label for the inactive tab.
tabBarStyle: Allows you to customize the background color, padding, and height of the tab bar.
tabBarLabelStyle:
Customizes the style of the tab label, such as font size and weight.
tabBarIcon: Adds custom icons to each tab.





Drawer Navigation Basic Navigation

1 `npm install @react-navigation/native @react-navigation/drawer react-native-screens react-native-safe-area-context`

2 screens/HomeScreen.js

```
import { View, Text } from "react-native";

export default function HomeScreen() {
  return (
    <View>
      <Text>Home Screen</Text>
    </View>
  );}
```

3 screens/ProfileScreen.js

```
import { View, Text } from "react-native";

export default function ProfileScreen() {
  return (
    <View>
      <Text>Profile Screen</Text>
    </View>
  );}
```

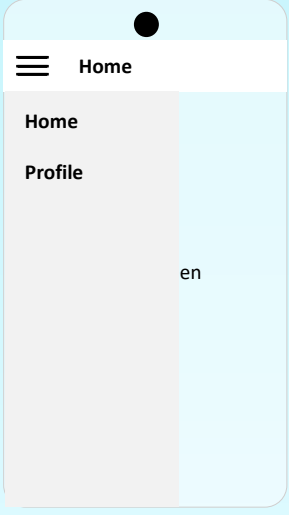
4 App.js

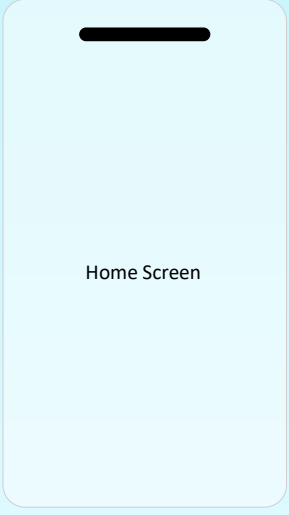
```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createDrawerNavigator } from "@react-navigation/drawer";
import HomeScreen from "./screens/HomeScreen";
import ProfileScreen from "./screens/ProfileScreen";

const Drawer = createDrawerNavigator(); //make instance of createDrawerNavigator
-------------------------------------------------------------------------------------------
export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator> //drawer navigation
        <Drawer.Screen name="Home" component={HomeScreen} /> //home screen route
        <Drawer.Screen name="Profile" component={ProfileScreen} /> //profile screen route
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

Step 1: Install the Required Libraries

To set up Drawer Navigation in your React Native project, you'll need to install two packages:

You can set up Drawer Navigation by using the createDrawerNavigator() function from @react-navigation/drawer





Drawer Navigation-II Custom Drawer Content

1 screens/CustomDrawerContent.js

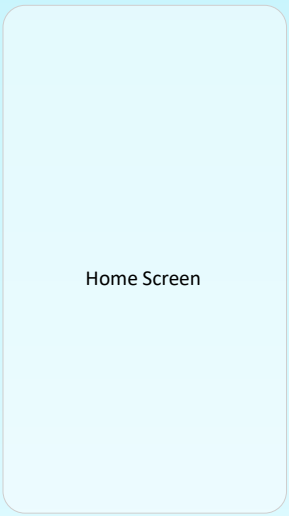
```
import React from "react";
import { View, Text, Button } from "react-native";

export default function CustomDrawerContent({ navigation }) {
  return (
    <View style={{ flex: 1, padding: 20, backgroundColor: "red" }}>
      <View>  <Text>Anjesh Kumar Ravi</Text></View>

      <Button title="Go to Home" onPress={() => navigation.navigate("Home")} />
      <Button title="Go to Settings" onPress={() => navigation.navigate("Settings")}/>
    </View>
  );
}
```

2 App.js

```
import React from "react";
import { View, Text } from "react-native";
import { NavigationContainer } from "@react-navigation/native";
import { createDrawerNavigator } from "@react-navigation/drawer";
import CustomDrawerContent from "./screens/CustomDrawerContent"; //import custom content
-------------------------------------------------------------------------------------------

function HomeScreen() {       //define screen for home Screen
  return ( <View> <Text>Home Screen</Text></View>);
}

function SettingsScreen() {   //define screen for Setting Screen
  return ( <View> <Text>Settings Screen</Text> </View>);
}
-------------------------------------------------------------------------------------------

const Drawer = createDrawerNavigator(); //initialize drawer Navigation

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator //render custom drawer content in drawerContent props
        drawerContent={(props) => <CustomDrawerContent {...props} />}>
        <Drawer.Screen name="Home" component={HomeScreen} /> //navigate to home screen
        <Drawer.Screen name="Settings" component={SettingsScreen} /> //navigate to setting
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

Step 1: Install the Required Libraries
2 : Create a Custom Drawer Content component
3 : Set Up the Drawer Navigator

Key Points:
Customization:The CustomDrawerContent replaces the default drawer layout.Add any components, styles, or navigation logic you want.
Props in CustomDrawerContent:navigation: Used to navigate to other screens.state: Contains the current navigation state.
Styling:Use StyleSheet to style your custom drawer to match your app theme.



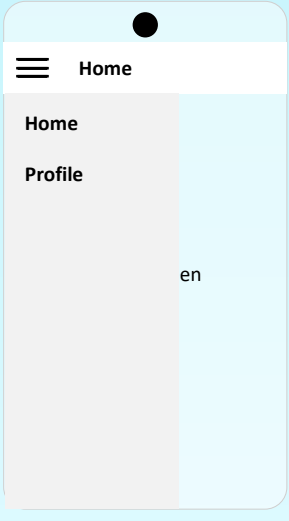
Drawer Navigation-III Adding Icons to Drawer Items

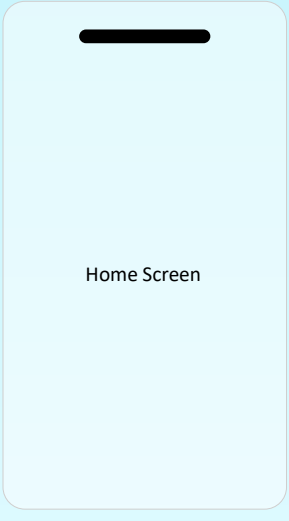
1 npm install react-native-vector-icons

2 App.js

```
import React from "react";
import { NavigationContainer } from "@react-navigation/native";
import { createDrawerNavigator } from "@react-navigation/drawer";
import Icon from "react-native-vector-icons/Ionicons"; //import icon
import HomeScreen from "./screens/HomeScreen";
import ProfileScreen from "./screens/ProfileScreen"; //import screen component

const Drawer = createDrawerNavigator(); //make instance of createdrawerNaigator
-------------------------------------------------------------------------------------------

export default function App() {
  return (
    <NavigationContainer>
      <Drawer.Navigator>                    //Drawer Navigator
-------------------------------------------------------------------------------------------
        <Drawer.Screen                       //Drawer Screen
          name="Home"
          component={HomeScreen}
          options={{
            drawerIcon: (color, size) => (   //Drawer item Icon
              <Icon name="home-outline" size={size} color={color} />
            ),
          }}
        />
-------------------------------------------------------------------------------------------
        <Drawer.Screen
          name="Profile"
          component={ProfileScreen}
          options={{
            drawerIcon: () => (
              <Icon name="person-outline" size={20} color={"red"} />
            ),
          }}
        />
-------------------------------------------------------------------------------------------
      </Drawer.Navigator>
    </NavigationContainer>
  );
}
```

Step 1: Install the Required Libraries
Then, you can customize each drawer item with an icon using options inside the Drawer.Screen.



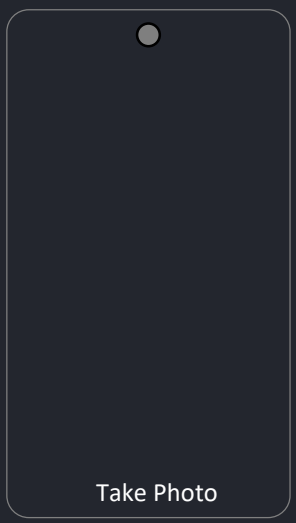


Property/Key	Short Description
drawerPosition	Sets the position of the drawer. Values: "left" (default) or "right".
drawerType	Determines how the drawer appears. Values: "front", "back", "slide", or "permanent".
drawerStyle	Customizes the style of the drawer container (e.g., width, background color).
overlayColor	Sets the overlay color for the drawer when it is open. Default: rgba(0, 0, 0, 0.5).
headerShown	Toggles the visibility of the header. Default: true.
swipeEdgeWidth	Specifies how far from the edge you can swipe to open the drawer. Default: 20.
swipeEnabled	Enables or disables swipe gestures to open/close the drawer. Default: true.
gestureHandlerProps	Passes additional props to the gesture handler.
drawerContent	A custom component to render the content of the drawer.
drawerContentOptions	Deprecated. Use drawerContent to customize drawer items instead.
screenOptions	Default options to pass to all screens inside the drawer navigator.
sceneContainerStyle	Style for the container wrapping each screen.
lazy	Whether to load screens lazily. Default: true.
unmountOnBlur	Unmounts screens when they are not focused. Default: false.
statusBarAnimation	Animation style for the status bar when the drawer is open. Values: "none", "slide", "fade".

1 npx expo install expo-camera

2 App.js

```
import React, { useState, useEffect, useRef } from "react";
import { View, Text, Button, Image } from "react-native";
import { Camera, CameraView } from "expo-camera"; //import Camera component

function CameraApp() {
  const [hasPermission, setHasPermission] = useState(null);
  const [photo, setPhoto] = useState(null);
  const cameraRef = useRef(null);

  // Ask for camera permission
  useEffect(() => {
    (async () => {
      const { status } = await Camera.requestCameraPermissionsAsync();
      setHasPermission(status === "granted");
    })();
  }, []);

  // Take a picture
  const takePicture = async () => {
    if (cameraRef.current) {
      const photoData = await cameraRef.current.takePictureAsync();
      setPhoto(photoData.uri);
    }
  };

  // Render messages based on permission state
  if (hasPermission === null) {
    return <Text>Requesting camera permission...</Text>;
  }
  if (hasPermission === false) {
    return <Text>No access to the camera</Text>;
  }

  return (
    <View style={{ flex: 1 }}>
      {!photo ? (
        <>
          <CameraView ref={cameraRef} style={{ flex: 1 }} />
          <Button title="Take Photo" onPress={takePicture} />
        </>
      ) : (
        <>
          <Image source={{ uri: photo }} style={{ flex: 1 }} />
          <Button title="Retake" onPress={() => setPhoto(null)} />
        </>
      )}
    </View>
  );
}

export default function App() {
  return <CameraApp />;
}
```



1) Install expo-camera:
2) Request Camera Permission: Use Camera.requestCameraPermissionsAsync() inside a useEffect to request permission when the component mounts.

React Native Location

1 `npx expo install expo-location`

2 App.js

```
import React, { useState, useEffect } from "react";
import { View, Text, Button } from "react-native";
import * as Location from "expo-location"; //import location component

const App = () => {
  const [location, setLocation] = useState(null); //initialize location state

  useEffect(() => {
    (async () => {
      let { status } = await Location.requestForegroundPermissionsAsync();
      if (status !== "granted") {
        alert("Permission to access location was denied");
        return;
      }
      let loc = await Location.getCurrentPositionAsync({});
      setLocation(loc);
    })();
  }, []); //ask for location permission

  return (
    <View style={{ padding: 20 }}>
      {location ? (
        <Text>
          Latitude: {location.coords.latitude}, Longitude:{" "}
          {location.coords.longitude}
        </Text>
      ) : (
        <Text>Loading location...</Text>
      )}
    </View>
  );
};

export default App;
```

To access device location in Expo

1) Install expo-location:
2) **Request Permissions:** The app requests permission to access the user's location using Location.requestForegroundPermissionsAsync(). If denied, it alerts the user.
3) Fetch Current Location: Once permission is granted, the user's current location is retrieved with Location.getCurrentPositionAsync().
4) Usage: The Location API provides an easy way to handle permissions and retrieve location data in Expo-managed React Native projects. The location object contains details like latitude and longitude.

Latitude :
longitude :

Latitude :
longitude :

React Native pushNotification

1 `npx expo install expo-notifications`

2 App.js

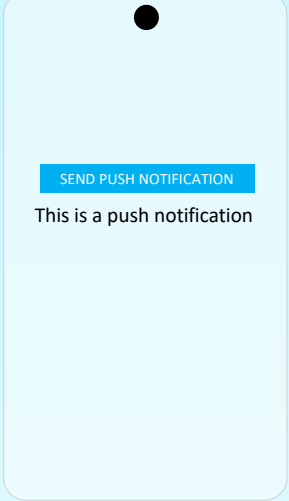
```
import React, { useState, useEffect } from "react";
import { View, Button, Text } from "react-native";
import * as Notifications from "expo-notifications";

const App = () => {
 const [notification, setNotification] = useState(false); // State to store notification data

  useEffect(() => {
    // Adding listener to detect when a notification is received
    const subscription = Notifications.addNotificationReceivedListener(
      (notification) => {
        setNotification(notification); // Update state with received notification
      }
    );

    // Cleanup function to remove the listener when component unmounts
    return () => subscription.remove();
  }, []); // Empty dependency array means this effect runs only once when the component
mounts

  // Function to trigger a push notification after a delay of 2 seconds
  const triggerPushNotification = async () => {
    await Notifications.scheduleNotificationAsync({
      content: {
        title: "Hello!", // Title of the notification
        body: "This is a push notification", // Body of the notification
      },
      trigger: { seconds: 2 }, // Trigger the notification after 2 seconds
    });
  };

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      {/* Button that triggers the push notification when pressed */}
      <Button title="Send Push Notification" onPress={triggerPushNotification}/>
      {/* Display the notification body when a notification is received */}
      {notification && <Text>{notification.request.content.body}</Text>}
    </View>
  );
};

export default App;
```

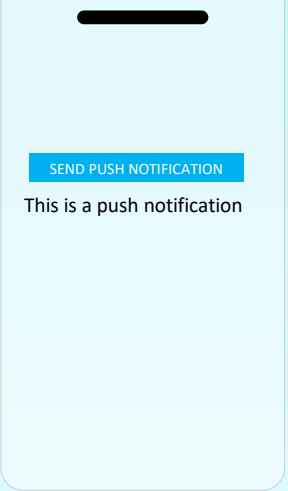
State Initialization: The notification state is initialized to false. It will hold the data of the received notification.

Effect Hook: The useEffect hook runs once when the component mounts, setting up a listener for received notifications. It also ensures the listener is removed when the component unmounts.

Trigger Notification: The triggerPushNotification function schedules a notification to appear 2 seconds after being triggered.

Rendering: The Button triggers the notification, and if a notification is received, its body is displayed in a Text element.





React Native fileSystem

1 `npx expo install expo-file-system`

2 App.js

```
import React, { useState, useEffect } from "react";
import { View, Button, Text } from "react-native";
import * as FileSystem from "expo-file-system"; // Importing Expo FileSystem API

const App = () => {
  // State to hold the content of the file after reading it
  const [fileContent, setFileContent] = useState("");

  // Function to write data to a file in the app's document directory
  const writeFile = async () => {
    // Defining the file URI (path) where the file will be stored
    const fileUri = FileSystem.documentDirectory + "example.txt";

    // Writing the string "Hello, Expo File System!" to the file at the specified URI
    await FileSystem.writeAsStringAsync(fileUri, "Hello, Expo File System!");

    // Showing an alert when the file is written
    alert("File written!");
  };

  // Function to read the content of the file from the document directory
  const readFile = async () => {
    // Defining the file URI (path) where the file is located
    const fileUri = FileSystem.documentDirectory + "example.txt";

    // Reading the content of the file at the specified URI
    const content = await FileSystem.readAsStringAsync(fileUri);

    setFileContent(content); // Updating the state to display the content of the file
  };

  return (
    <View
      style={{ flex: 1, justifyContent: "center", alignItems: "center",padding: 20,}}>
      <Button title="Write File" onPress={writeFile} />
      <Button title="Read File" onPress={readFile} />
      {fileContent && <Text>{fileContent}</Text>}
    </View>
  );
};

export default App;
```

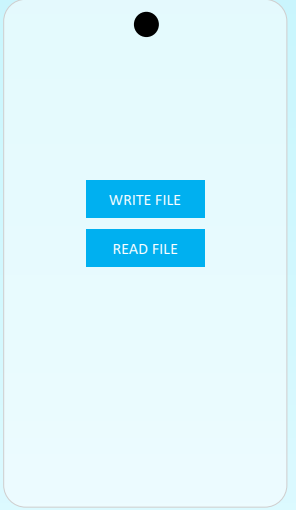
Imports: expo-file-system is imported to use its functionality for reading and writing files.

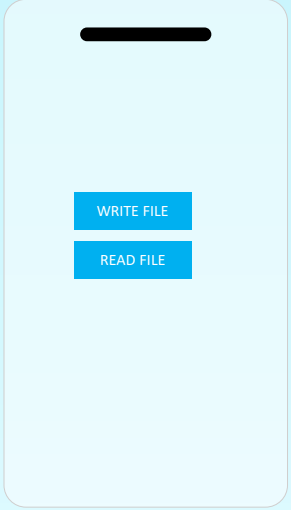
State (fileContent): This stores the content of the file once it is read.

writeFile Function: This writes a string to a file called example.txt in the document directory and displays an alert upon completion.

readFile Function: This reads the content of example.txt from the document directory and updates the fileContent state

UI: Two buttons are provided for writing and reading files, and the content of the file (if it exists) is displayed in a Text element.





React Native Sensors-Accelerometer

1 `npx expo install expo-sensors`

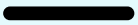
2 App.js

```
// Import React and necessary modules
import React, { useState, useEffect } from "react";
import { View, Text } from "react-native";
import { Accelerometer } from "expo-sensors";

const App = () => {
  // State to store accelerometer data
  const [sensorData, setSensorData] = useState({ x: 0, y: 0, z: 0 });

  useEffect(() => {
    // Start listening to accelerometer data
    const subscription = Accelerometer.addListener((data) => {
      setSensorData(data);
    });

    // Clean up the subscription when the component unmounts
    return () => subscription.remove();
  }, []);

  return (
    <View
      style={{
        flex: 1,
        justifyContent: "center",
        alignItems: "center",
        padding: 20,
      }}
    >
      <Text>x: {sensorData.x.toFixed(2)}</Text>
      <Text>y: {sensorData.y.toFixed(2)}</Text>
      <Text>z: {sensorData.z.toFixed(2)}</Text>
    </View>
  );
};

export default App;
```

expo-sensors is an Expo library that provides access to hardware sensors like the accelerometer, gyroscope, magnetometer, and barometer. It allows apps to detect motion, orientation, magnetic fields, and atmospheric pressure. Developers can subscribe to sensor updates, adjust data frequency, and manage listeners for efficient use. Common use cases include fitness tracking, gaming, AR, and navigation.

Initialize State: Store x, y, z values for accelerometer data.
Subscribe: Add a listener to receive accelerometer updates.
Update State: Update the x, y, z values as new data comes in.
Display: Show the accelerometer values on the screen.
Cleanup: Remove the listener when the component unmounts.

x:02609823084
y:0.170098405
z: 0.92.7932749



x:02609823084
y:0.170098405
z: 0.92.7932749

1 App.js

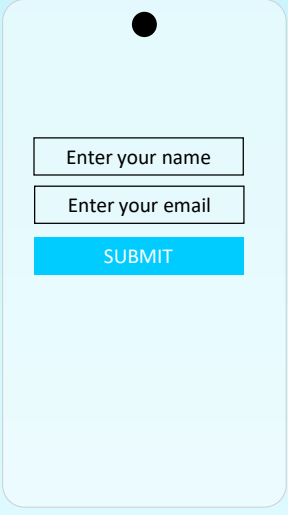
```
import React, { useState } from "react";
import { View, TextInput, Button, Text, StyleSheet } from "react-native";

const App = () => {
  const [name, setName] = useState(""); // State to store the name input
  const [email, setEmail] = useState(""); // State to store the email input
  const [submitted, setSubmitted] = useState(false);//State to check if the form is submitted

  // Function to handle the form submission
  const handleSubmit = () => {
    setSubmitted(true); // Set submitted to true when the button is pressed
  };

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input}
        placeholder="Enter your name" // Placeholder text for name input
        value={name} // Value bound to the name state
        onChangeText={(text) => setName(text)} // Update the name state when text changes
      />
      <TextInput
        style={styles.input}
        placeholder="Enter your email"
        keyboardType="email-address" // Use email keyboard layout
        value={email} // Value bound to the email state
        onChangeText={(text) => setEmail(text)} // Update the email state when text changes
      />
      <Button title="Submit" onPress={handleSubmit} />
      {submitted && (<Text style={styles.result}> Name: {name}, Email: {email}</Text>)}
    </View>);
};

const styles = StyleSheet.create({
  container: {flex: 1, justifyContent: "center", padding: 20,},
  input: {borderWidth: 1, padding: 10, marginVertical: 10,},
  result: {marginTop: 20, fontSize: 16,},
});

export default App; // Export the App component as the default export
```

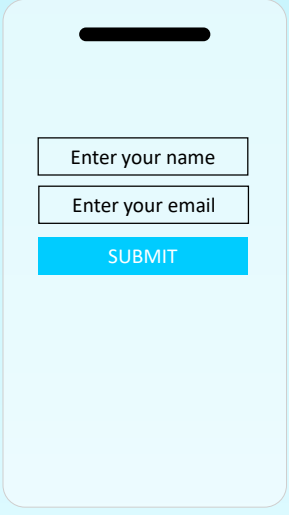
State Management:name, email, and submitted are managed using the useState hook.name and email store the user's input, while submitted tracks if the form has been submitted.

Form Handling: Two TextInput components allow users to input their name and email.A Button triggers the handleSubmit function, which sets submitted to true.

Conditional Rendering:After submission, a Text element displays the entered name and email, rendered only when submitted is true.

Styling:The StyleSheet defines basic styling for layout (container), input fields (input), and the result text (result).





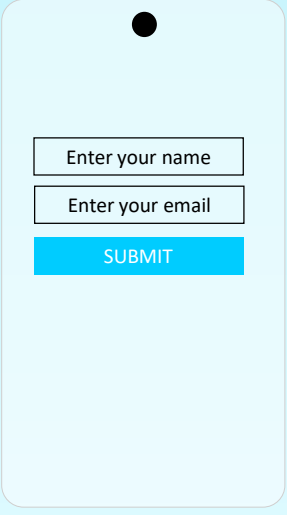
React Native Formik for Form Management

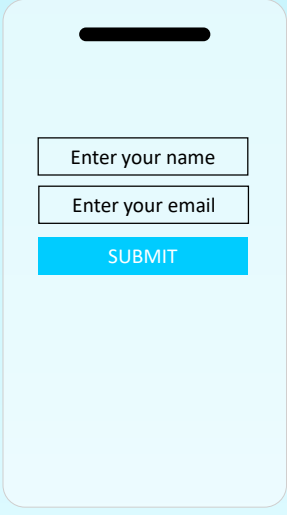
1 `npm install formik`

2 App.js

```
import React from "react";
import { View, TextInput, Button, StyleSheet } from "react-native";
import { Formik } from "formik";

const App = () => {
  return (
    // Formik is used to manage form state and handle form submission
    <Formik
      initialValues={{ name: "", email: "" }} // Initial values for the form inputs
      onSubmit={(values) => alert(JSON.stringify(values))} // Action to on form submission
    >
      {({ handleChange, handleBlur, handleSubmit, values }) => (
        <View style={styles.container}>
          <TextInput
            style={styles.input}
            placeholder="Enter your name"
            onChangeText={handleChange("name")} // Updates the 'name' value in the form
            onBlur={handleBlur("name")} // Marks the 'name' field as touched
            value={values.name} // Binds the value of the input to the 'name' field
          />
          <TextInput
            style={styles.input}
            placeholder="Enter your email"
            keyboardType="email-address" // Sets the keyboard type for email input
            onChangeText={handleChange("email")} // Updates the 'email' value in the form
            onBlur={handleBlur("email")} // Marks the 'email' field as touched
            value={values.email} // Binds the value of the input to the 'email' field
          />
          <Button title="Submit" onPress={handleSubmit} />{" "}
        </View>
      )}
    </Formik>
  );
};

const styles = StyleSheet.create({
  container: { flex: 1, justifyContent: "center", padding: 20,},
  input: {borderWidth: 1, padding: 10, marginVertical: 10,},
});

export default App;
```

Form State Management: Formik provides a centralized way to manage the state of all form inputs. For example, it keeps track of values entered in form fields, whether a field has been touched, and any validation errors.
Validation: It allows for easy validation of form inputs, either through custom logic or by integrating with validation libraries like Yup. This ensures that user inputs meet specific criteria before submission.
Submission Handling: Formik makes it straightforward to handle form submissions. It provides a method to collect form data and trigger a specific action (e.g., sending data to an API).
Flexibility: Formik works with controlled components, meaning developers have full control over form behavior while benefiting from the library's utilities.
Ease of Use: By abstracting away common form-related tasks, it reduces boilerplate code, making forms easier to implement and maintain.





React Native Input Validation with Yup

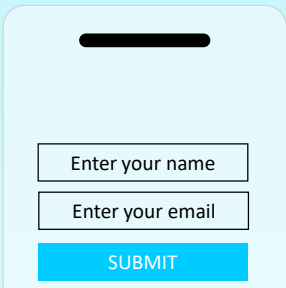
1 `npm install yup`

2 App.js

```
import React from "react";
import { View, TextInput, Button, Text, StyleSheet } from "react-native";
import { Formik } from "formik";
import * as Yup from "yup";

// Validation schema with Yup
const validationSchema = Yup.object({
  name: Yup.string().required("Name is required"),
  email: Yup.string().email("Invalid email").required("Email is required"),
});

const App = () => {
  return (
    <Formik
      initialValues={{ name: "", email: "" }}
      validationSchema={validationSchema}
      onSubmit={(values) => alert(JSON.stringify(values))}
    >
      {({handleChange, handleBlur, handleSubmit, values, errors, touched,}) => (
        <View style={styles.container}>
          <TextInput
            style={{ flex: 1, justifyContent: "center", padding: 20 }}
            onChangeText={handleChange("name")}
            onBlur={handleBlur("name")} value={values.name}
          />
          {touched.name && errors.name && (
              <Text style={{ color: "red", fontSize: 12 }}>{errors.name}</Text>)}

          <TextInput
            style={ borderWidth: 1, padding: 10, marginBottom: 10 },}
            keyboardType="email-address"
            onChangeText={handleChange("email")}
            onBlur={handleBlur("email")} value={values.email}
          />
          {touched.email && errors.email && (
              <Text style={{ color: "red", fontSize: 12 }}>{errors.email}</Text>
          )}
          <Button title="Submit" onPress={handleSubmit} />
        </View>
      )}</Formik> )};

export default App;
```

Yup is a JavaScript schema validation library commonly used with form handling in React and React Native. It helps define rules for validating data, such as ensuring fields are required, have a specific format (e.g., email), or meet custom conditions.

Schema-based: You define a schema (structure) for your data and use it to validate inputs.
Validation Rules: Offers built-in rules like required(), min(), max(), email(), etc., to enforce constraints on form fields.
Chaining: You can chain validation methods to apply multiple checks on a field.
Error Messages: You can specify custom error messages for validation failures.
Asynchronous Validation: Supports async validation, such as checking data with an API or a database.



1 App.js

```
import React, { useState, useEffect } from "react"; // Import necessary hooks from React
import { View, Text, FlatList, StyleSheet, ActivityIndicator,} from "react-native";

const App = () => {
  const [data, setData] = useState([]); // State for storing fetched data
  const [loading, setLoading] = useState(true); // State for handling loading status

   useEffect(() => { // Fetch data when the component mounts
     const fetchData = async () => {
       try {
         const response = await fetch("https://jsonplaceholder.typicode.com/posts");
         const json = await response.json(); // Parse the response as JSON
         setData(json); // Save the data in state
       } catch (error) {
         console.error(error); // Log errors, if any
       } finally {
         setLoading(false); // Stop the loading indicator
       }
     };
     fetchData(); // Call the fetch function
   }, []); // Empty dependency array ensures it runs only once

  return (
    <View style={styles.container}>
      {loading ? (<ActivityIndicator size="large" color="#0000ff" />) : (
        <FlatList
          data={data} // Data source for the FlatList
          keyExtractor={(item) => item.id.toString()} // Unique key for each item
          renderItem={({ item }) => (
            <View style={styles.item}>
              <Text style={styles.title}>{item.title}</Text>
            </View>
          )}
        />
      )}
    </View>
  );
};

const styles = StyleSheet.create({
  container: {flex: 1, padding: 20,},
  item: { marginVertical: 10, padding: 15, backgroundColor: "#f9f9f9", borderRadius: 5,},
  title: { fontSize: 16, fontWeight: "bold",},
});
export default App;
```

The fetch API is a built-in JavaScript function to make HTTP requests.

Request: It sends a GET request to the URL https://jsonplaceholder.typicode.com/posts to fetch data.

Response: The server responds with raw data.

Parsing: The response.json() method converts the raw data into a usable JavaScript object or array (JSON format).

Error Handling: A try-catch block ensures errors (e.g., network issues) are caught and logged.

React Native Fetch Data Using Axios

1 `npm install axios`

2 App.js

```
import React, { useState, useEffect } from "react";
import { View, Text, FlatList, StyleSheet } from "react-native";
import axios from "axios"; // Importing Axios library for HTTP requests

const App = () => {
   const [data, setData] = useState([]); // State to store the fetched data
-----------------------------------------------------------------------------------------
  // useEffect hook to fetch data when the component mounts
  useEffect(() => {
    const fetchData = async () => {
      try {
        // Making a GET request to fetch posts from a placeholder API
        const response = await axios.get("https://jsonplaceholder.typicode.com/posts");
        setData(response.data); // Updating state with the fetched data
      } catch (error) {
        console.error(error); // Logging any errors to the console
      }};

    fetchData(); // Calling the function to fetch data
  }, []); // Empty dependency array ensures this effect runs only once
-----------------------------------------------------------------------------------------
  return (
    <View style={styles.container}>
      <FlatList
        data={data} // The data to be displayed
        keyExtractor={(item) => item.id.toString()} // Unique key for each item
        renderItem={({ item }) => (
          <View style={styles.item}><Text style={styles.title}>{item.title}</Text></View>
        )}/>
    </View>
  );};
-----------------------------------------------------------------------------------------

const styles = StyleSheet.create({
  container: {flex: 1, padding: 20,},
  item: { marginVertical: 10, padding: 15, backgroundColor: "#f9f9f9", borderRadius: 5,},
  title: { fontSize: 16, fontWeight: "bold",},
});

export default App;
```

State and Effect:useState is used to store the data fetched from the API.useEffect is used to fetch data when the component is first rendered.

Fetching Data:Axios is used to fetch data from an API endpoint.Error handling is included to log any issues that arise during the fetch process.

Rendering the List:FlatList is ideal for rendering long lists efficiently.Each list item is displayed using the renderItem function.

1 npm install @tanstack/react-query @tanstack/react-query-devtools axios

2 App.js

```
import React from "react";
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";
import MainScreen from "./assets/MainScreen";

const queryClient = new QueryClient(); //initialize QueryClient for caching & data fetching

export default function App() {
  return (
    <QueryClientProvider client={queryClient}> //provide Query to app wrap main screen
      <MainScreen />
    </QueryClientProvider>
  );
}
```

3 Assets/MainScreen.js

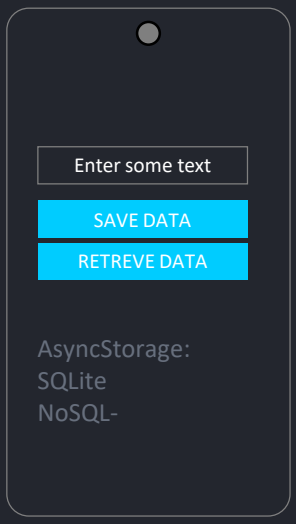
```
import React from "react";
import { View, Text, FlatList, StyleSheet } from "react-native";
import { useQuery } from "@tanstack/react-query";
import axios from "axios"; //import all require component and library

const fetchPosts = async () => {            //in this function fetch data and return
  const response = await axios.get("https://jsonplaceholder.typicode.com/posts");
  return response.data;
};
-------------------------------------------------------------------------------------

export default function MainScreen() {
  const { data, isLoading, error } = useQuery({queryKey: ["posts"],queryFn: fetchPosts,});

  if (isLoading) return <Text style={styles.message}>Loading...</Text>;
  if (error) return <Text style={styles.message}>Error fetching data</Text>;

// useQuery Purpose: Fetch and cache data.
// queryKey: A unique identifier for the query.
// queryFn: A function that returns a promise (usually fetching data).
// Features: Automatically caches results, handles loading and error states.
-------------------------------------------------------------------------------------
  return (
    <View style={styles.container}>
      <FlatList
        data={data}
        keyExtractor={(item) => item.id.toString()}
        renderItem={({ item }) => (
          <View style={styles.item}>
            <Text style={styles.title}>{item.title}</Text>
          </View>
        )}
     />
    </View>
  );}
-------------------------------------------------------------------------------------
const styles = StyleSheet.create({
  container: {flex: 1, padding: 20,},
  message: {textAlign: "center", marginTop: 50, fontSize: 18, color: "gray",},
  item: {marginVertical: 10, padding: 15, backgroundColor: "#f9f9f9", borderRadius: 5,},
  title: {fontSize: 16, fontWeight: "bold",},
});
```

React Native AsyncStorage

1 `npm install @react-native-async-storage/async-storage`

2 **StorageService.js**

```
// StorageService.js
import AsyncStorage from "@react-native-async-storage/async-storage";

export const saveData = async (key, value) => {
  try {
    await AsyncStorage.setItem(key, value);
    alert("Data saved!");
  } catch (error) {
    console.error("Error saving data", error);
  }
}; //saved data using AsycnStorage.setItem method (key , value)

export const retrieveData = async (key) => {
  try {
    const value = await AsyncStorage.getItem(key);
    return value;
  } catch (error) {
    console.error("Error retrieving data", error);
    return null;
  }
}; //read saved data using AsyncStorage.getItem method (key)
```



3 **App.js**

```
import React, { useState } from "react";
import { View, TextInput, Button, Text, StyleSheet } from "react-native";
import { saveData, retrieveData } from "./StorageService"; //import custom method

const App = () => {
  const [input, setInput] = useState(""); //hold input value
  const [storedData, setStoredData] = useState(""); //hold stored data for show in alert

  const handleSave = () => {
    saveData("userInput", input); //save data using your custom method (key, value)
  };

  const handleRetrieve = async () => {
    const value = await retrieveData("userInput");
    if (value !== null) {
      setStoredData(value);
    }
  }; //read data using custom method retrieveData (key)

  return (
    <View style={styles.container}>
      <TextInput
        style={styles.input} placeholder="Enter some text" value={input}
        onChangeText={(text) => setInput(text)}/>
      <Button title="Save Data" onPress={handleSave} />
      <Button title="Retrieve Data" onPress={handleRetrieve} />
      {storedData ? (<Text style={styles.result}>Stored Data: {storedData}</Text>) : null}
    </View>
  );
};
const styles = StyleSheet.create({
  container: {flex: 1, justifyContent: "center", padding: 20,},
  input: {borderWidth: 1, padding: 10, marginVertical: 10,},
  result: {marginTop: 20, fontSize: 16, fontWeight: "bold",},
});
export default App;
```

AsyncStorage: Best for simple key-value storage.SQLite: Ideal for relational database needs.Realm: Great for NoSQL-like storage with high performance.

1 App.js

```
import React, { useRef, useEffect } from "react";
import { Animated, StyleSheet, Text, View } from "react-native";

const FadeInExample = () => {
  // Create a reference for the animated value (initial opacity: 0)
  const fadeAnim = useRef(new Animated.Value(0)).current;
-----------------------------------------------------------------------------------------------

  useEffect(() => {
    // Animate the opacity value from 0 to 1 over 2 seconds
    Animated.timing(fadeAnim, {
      toValue: 1, // Final opacity
      duration: 2000, // Animation duration in milliseconds
      useNativeDriver: true, // Use native driver for better performance
    }).start(); // Start the animation
  }, [fadeAnim]); // Run effect whenever fadeAnim changes (though it won't in this case)
-----------------------------------------------------------------------------------------------

  return (
    <View style={styles.container}>
        <Animated.View style={{ ...styles.box, opacity: fadeAnim }}>
        <Text style={styles.text}>Fade In!</Text>
      </Animated.View>
    </View>
  );
};
-----------------------------------------------------------------------------------------------

const styles = StyleSheet.create({
  container: {
    flex: 1, // Take up the full screen
    justifyContent: "center", // Center content vertically
    alignItems: "center", // Center content horizontally
  },
  box: {
    padding: 20, // Add padding inside the box
    backgroundColor: "#61dafb", // Light blue background
    borderRadius: 10, // Rounded corners
  },
  text: {
    fontSize: 20, color: "#fff",},
});

export default FadeInExample;
```

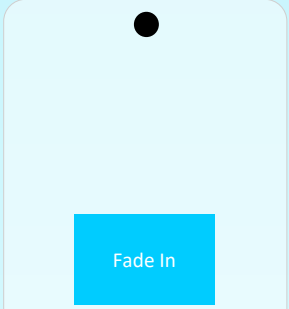
Setting up

You don't need to install anything extra when using Animated as it's built into React Native

Concepts to focus on:

Animated.Value: Stores the initial value.
Animated.timing: Creates a timed animation.
useNativeDriver: Ensures the animation runs on the UI thread for better performance.

Exercises:Try animating properties like translateX or scale for movement and resizing.Combine multiple animations using Animated.parallel or Animated.sequence.



1 App.js

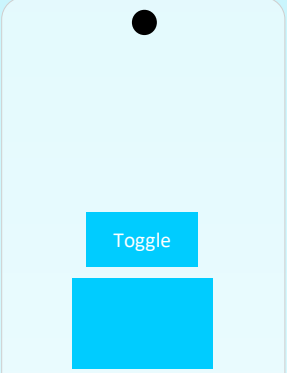
```
import React, { useState } from "react";
import {LayoutAnimation, StyleSheet, Text, TouchableOpacity, View,} from "react-native";

const LayoutAnimationExample = () => {
  const [expanded, setExpanded] = useState(false); // State to toggle box size

  const toggleBox = () => {
    // Animate layout changes using the 'spring' preset
    LayoutAnimation.configureNext(LayoutAnimation.Presets.spring);
    setExpanded(!expanded); // Toggle the box size
  };
-----------------------------------------------------------------------------------------------

  return (
    <View style={styles.container}>
      {/* Button to trigger box toggle */}
      <TouchableOpacity onPress={toggleBox} style={styles.button}>
        <Text style={styles.buttonText}>Toggle</Text>
      </TouchableOpacity>

      {/* Box that changes size when toggled */}
      <View style={[styles.box, expanded && styles.expandedBox]} />
    </View>
  );
};
-----------------------------------------------------------------------------------------------

const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center", // Center content on the screen
  },
  button: {
    marginBottom: 20,
    padding: 10,
    backgroundColor: "#61dafb", // Button color
    borderRadius: 5,
  },
  buttonText: {color: "#fff", fontSize: 16,},
  box: {width: 100, height: 100, backgroundColor: "#61dafb",},
  expandedBox: {width: 200, height: 200,},
});

export default LayoutAnimationExample;
```

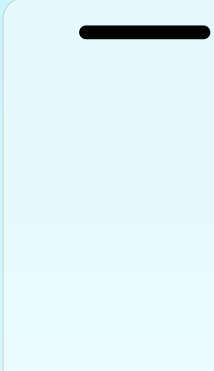
LayoutAnimation and Transitions
LayoutAnimation is used for animating layout changes (e.g., resizing or reordering components).

Concepts:
LayoutAnimation.configureNext:
Triggers a layout animation on state updates.

Presets:
easeInEaseOut, linear, or spring provide built-in animations.

Exercises:Add more components and animate their layout when toggling.Experiment with custom animation configurations using LayoutAnimation.create





1 registerPushNotifications.js

```
import { Alert, Platform } from "react-native";
import * as Notifications from "expo-notifications";
import * as Device from "expo-device";



// Register for push notifications and get the token
export async function registerForPushNotificationsAsync() {
  if (Device.isDevice) {
    const { status: existingStatus } =
      await Notifications.getPermissionsAsync();
    let finalStatus = existingStatus;



    // Request permission if not already granted
    if (existingStatus !== "granted") {
      const { status } = await Notifications.requestPermissionsAsync();
      finalStatus = status;
    }




    if (finalStatus !== "granted") {
      Alert.alert("Failed to get push token for push notifications!");
      return;
    }



    // Get the Expo push token
    const token = (await Notifications.getExpoPushTokenAsync()).data;
    console.log("Push Notification Token:", token);
    Alert.alert("Push Token", token);




    // Android: Set notification channel
    if (Platform.OS === "android") {
      await Notifications.setNotificationChannelAsync("default", {
        name: "default",
        importance: Notifications.AndroidImportance.MAX,
        vibrationPattern: [0, 250, 250, 250],
        lightColor: "#FF231F7C",
      });
    }



    return token;
  } else {
    Alert.alert("Must use physical device for Push Notifications");
  }
}
```

2 NotificationHandler.js

```
import * as Notifications from "expo-notifications";

// Set notification behavior
Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});
```

3 NotificationListeners.js

```
import { Alert } from "react-native";
import * as Notifications from "expo-notifications";

export function setupNotificationListeners() {
  // Listener for when a notification is received while the app is foregrounded
  const notificationListener = Notifications.addNotificationReceivedListener(
    (notification) => {
      Alert.alert(
        "Notification Received",
        `Title: ${notification.request.content.title}\nBody:
${notification.request.content.body}`
      );
    }
  );

  // Listener for when a user interacts with a notification
  const responseListener =
    Notifications.addNotificationResponseReceivedListener((response) => {
      Alert.alert(
        "Notification Tapped",
        `Data: ${JSON.stringify(response.notification.request.content.data)}`
      );
    });

  // Cleanup listeners
  return () => {
    Notifications.removeNotificationSubscription(notificationListener);
    Notifications.removeNotificationSubscription(responseListener);
  };
}
```

4 sendTestNotification.js

```
import * as Notifications from "expo-notifications";

// Schedule a test notification
export async function sendTestNotification() {
  await Notifications.scheduleNotificationAsync({
    content: {
      title: "📬 Test Notification!",
      body: "This is a test notification from your app.",
      data: { testData: "Some data to test" },
      sound: true,
    },
    trigger: { seconds: 2 }, // Delayed by 2 seconds
  });
}
```

5 App.js

```
import React, { useEffect } from "react";
import { Button, View } from "react-native";
import { registerForPushNotificationsAsync } from "./registerPushNotifications";
import { setupNotificationListeners } from "./NotificationListeners";
import { sendTestNotification } from "./sendTestNotification";
import "./NotificationHandler"; // Importing the notification handler setup

export default function App() {
  useEffect(() => {
    registerForPushNotificationsAsync();
    const cleanupListeners = setupNotificationListeners();

    return cleanupListeners;
  }, []);

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Button
        title="Test Push Notification"
        onPress={async () => {
          await sendTestNotification();
        }}
      />
    </View>
  );
}
```

Key Features in the Code:

Notification Handler:Notifications.setNotificationHandler defines how notifications are handled when received (e.g., showing alerts, playing sounds).

Register for Push Notifications: registerForPushNotificationsAsync checks and requests permissions for push notifications.If permissions are granted, it retrieves the Expo Push Token and configures a notification channel for Android.

Foreground and Interaction Listeners: Notifications. addNotificationReceivedListener listens for incoming notifications when the app is in the foreground and displays an alert with the notification's title and body.Notifications.addNotificationResponseReceivedListener listens for user interactions with notifications and logs the associated data.

T**est Notification**: sendTestNotification schedules a notification to test the functionality.

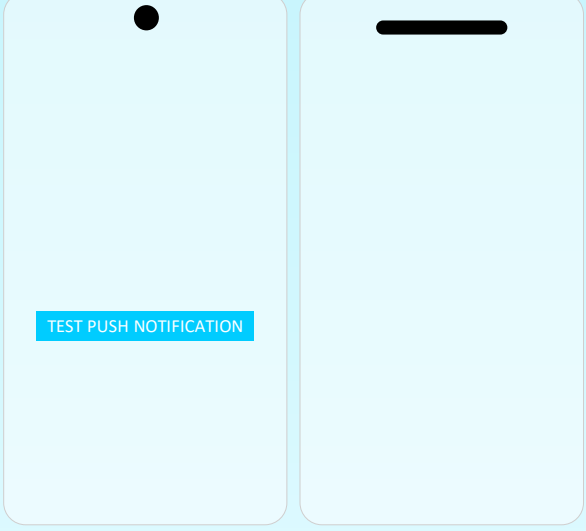
Button for Triggering Test Notification:A button triggers the test notification for demonstration purposes.

This React Native app uses Expo to handle push notifications. It:

Registers for Push Notifications:
Requests permissions, retrieves the Expo push token, and sets up an Android notification channel.

Listens for Notifications:
Handles notifications received in the foreground and user interactions with alerts.

Schedules Test Notifications:
A button triggers a test notification with a 2-second delay.



1 App.js

```
import React, { useEffect } from "react";
import { Button, Alert, Platform, View } from "react-native";
import * as Notifications from "expo-notifications";
import * as Device from "expo-device";

Notifications.setNotificationHandler({
  handleNotification: async () => ({
    shouldShowAlert: true,
    shouldPlaySound: true,
    shouldSetBadge: false,
  }),
});

export default function App() {
  useEffect(() => {
    registerForPushNotificationsAsync();

    // Listener for when a notification is received while the app is foregrounded
    const notificationListener = Notifications.addNotificationReceivedListener(
      (notification) => {
        Alert.alert(
          "Notification Received",
          `Title: ${notification.request.content.title}\nBody: ${notification.request.content.body}`
        );
      });

    // Listener for when a user interacts with a notification
    const responseListener =
      Notifications.addNotificationResponseReceivedListener((response) => {
        Alert.alert(
          "Notification Tapped",
          `Data: ${JSON.stringify(response.notification.request.content.data)}`
        );
      });

    return () => {
      Notifications.removeNotificationSubscription(notificationListener);
      Notifications.removeNotificationSubscription(responseListener);
    };
  }, []);

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Button
        title="Test Push Notification"
        onPress={async () => {
          await sendTestNotification();
        }}/>
    </View>
  );}

// Register for push notifications and get the token
async function registerForPushNotificationsAsync() {
  if (Device.isDevice) {
    const { status: existingStatus } =
      await Notifications.getPermissionsAsync();
    let finalStatus = existingStatus;

    if (existingStatus !== "granted") {// Request permission if not already granted
      const { status } = await Notifications.requestPermissionsAsync();
      finalStatus = status;
    }

    if (finalStatus !== "granted") {
      Alert.alert("Failed to get push token for push notifications!");
      return;
    }

    const token = (await Notifications.getExpoPushTokenAsync()).data; // Get the Expo push token
    console.log("Push Notification Token:", token);
    Alert.alert("Push Token", token);

        if (Platform.OS === "android") {// Android: Set notification channel
        await Notifications.setNotificationChannelAsync("default", {
        name: "default",
        importance: Notifications.AndroidImportance.MAX,
        vibrationPattern: [0, 250, 250, 250],
        lightColor: "#FF231F7C",
      });}

    return token;
  } else {
    Alert.alert("Must use physical device for Push Notifications");
  }}

async function sendTestNotification() {// Schedule a test notification
  await Notifications.scheduleNotificationAsync({
    content: {
      title: "🛍 Test Notification!",
      body: "This is a test notification from your app.",
      data: { testData: "Some data to test" },
      sound: true,
    },
    trigger: { seconds: 2 }, // Delayed by 2 seconds
  });}
```

1 npm install react-native-reanimated react-native-gesture-handler

2 babel.config.js

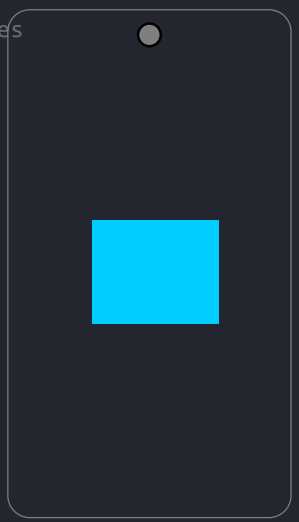
```
module.exports = {
  presets: ["babel-preset-expo"],
  plugins: ["react-native-reanimated/plugin"], //add this code in your babel.config.js file
};
```

3 App.js

```
import React from "react";
import { GestureHandlerRootView } from "react-native-gesture-handler";
import DraggableBox from "./DraggableBox";
// Replace with your component's path

export default function App() {
  return (
    <GestureHandlerRootView style={{ flex: 1 }}> //wrap component in GestureHandlerRootView
      <DraggableBox />
    </GestureHandlerRootView>
  );
}
```

4 DraggableBox.js

```
import React from "react";
import { StyleSheet, View } from "react-native";
import Animated, { useSharedValue, useAnimatedStyle, withSpring,
} from "react-native-reanimated"; // Import Reanimated for animations
import { PanGestureHandler } from "react-native-gesture-handler";

const DraggableBox = () => {
  const translateX = useSharedValue(0); // Shared values to store the X and Y translation
  const translateY = useSharedValue(0);

  // Animated style to update the transform property based on shared values
  const animatedStyle = useAnimatedStyle(() => ({
    transform: [
      { translateX: translateX.value }, // Apply horizontal movement
      { translateY: translateY.value }, // Apply vertical movement
    ],
  }));

  // Gesture handler to update translation values on drag
  const onGestureEvent = ({ nativeEvent }) => {
    translateX.value = nativeEvent.translationX; // Update X translation
    translateY.value = nativeEvent.translationY; // Update Y translation
  };

  return (
    <View style={styles.container}>
      <PanGestureHandler onGestureEvent={onGestureEvent}>
        <Animated.View style={[styles.box, animatedStyle]} />
      </PanGestureHandler>
    </View>
  );
};

const styles = StyleSheet.create({
  container: {flex: 1, justifyContent: "center", alignItems: "center",},
  box: {width: 100, height: 100, backgroundColor: "#61dafb", borderRadius: 10,},  });
export default DraggableBox; // Export the component
```



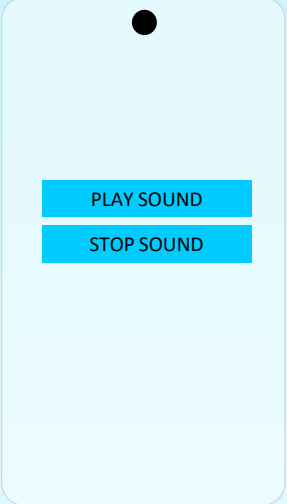
1 npx expo install expo-av

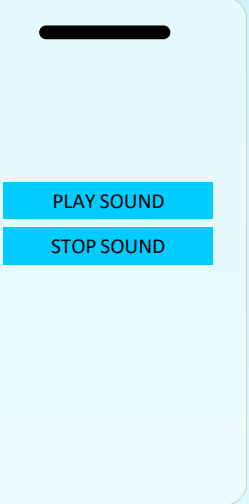
2 App.js

```
import React, { useState } from "react";
import { View, Button, Text } from "react-native";
import { Audio } from "expo-av";

export default function AudioPlayer() {
  const [sound, setSound] = useState();

  async function playSound() {
    // Load the sound
    const { sound } = await Audio.Sound.createAsync({
      uri: "https://www.soundhelix.com/examples/mp3/SoundHelix-Song-1.mp3",
    });
    setSound(sound);

    // Play the sound
    await sound.playAsync();
  }

  async function stopSound() {
    if (sound) {
      await sound.stopAsync();
    }
  }

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Button title="Play Sound" onPress={playSound} />
      <Button title="Stop Sound" onPress={stopSound} />
    </View>
  );
}
```

Install
expo-av Run this command in your project directory:

Playing Audio
You can play audio files (local or remote) using Audio.Sound from expo-av





React Native Video

1 `npx expo install expo-av`

2 App.js

```
import React from "react";
import { StyleSheet, View } from "react-native";
import { Video } from "expo-av";

// Video player component
export default function VideoPlayer() {
  return (
    <View style={styles.container}>
      <Video
        source={{ uri: "https://www.w3schools.com/html/mov_bbb.mp4" }} // Video source
        rate={1.0} // Playback speed
        volume={1.0} // Volume level
        isMuted={false} // Audio on
        resizeMode="cover" // Fit video to container
        shouldPlay // Autoplay video
        useNativeControls // Show video controls
        style={styles.video} // Video styling
      />
    </View>
  );
}

// Styles
const styles = StyleSheet.create({
  container: {
    flex: 1,
    justifyContent: "center",
    alignItems: "center",
    backgroundColor: "#000", // Black background
  },
  video: {
    width: 300,
    height: 200,
  },
});
```

Key Notes:

expo-av: A library for audio and video playback in React Native.

source: Accepts a URI or local file for video content.

resizeMode: Determines how the video fits the container (cover, contain, etc.).

useNativeControls: Enables built-in video controls.

shouldPlay: Makes the video autoplay on load.

React Native Image Picker

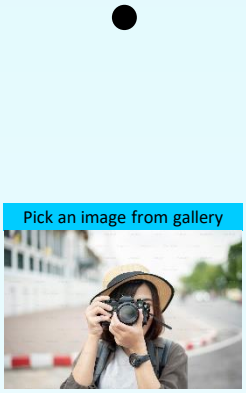
1 `npx expo install expo-image-picker`

2 App.js

```
import React, { useState } from "react";
import { View, Button, Image, Text } from "react-native";
import * as ImagePicker from "expo-image-picker";

export default function ImagePickerExample() {
  // State to store the selected image URI
  const [image, setImage] = useState(null);

  const pickImage = async () => {
    // Request permission to access the media library
    const { status } = await ImagePicker.requestMediaLibraryPermissionsAsync();
    if (status !== "granted") {
      alert("Sorry, we need camera roll permissions to make this work!");
      return;
    }

    // Open the image library to pick an image
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images, // Only images
      allowsEditing: true, // Allow editing the image
      aspect: [6, 6], // Aspect ratio for editing (square)
      quality: 1, // Image quality (1 = high)
    });

    // If the user picks an image, save the URI in state
    if (!result.canceled) {
      setImage(result.assets[0].uri);
    }
  };

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      {/* Button to trigger image picker */}
      <Button title="Pick an image from gallery" onPress={pickImage} />

      {/* Display the selected image if available */}
      {image && (
        <Image source={{ uri: image }} style={{ width: 200, height: 200 }} />
      )}
    </View>
  );
}
```

Key Notes:

State Explanation: Added to explain why useState is being used.
Permission Check: Clarified that permission is required to access the media library.
Picker Options: Described options like mediaTypes, allowsEditing, and aspect.
Condition Check: Noted what happens when an image is successfully selected.
UI Comments: Clarified the role of the Button and Image components.

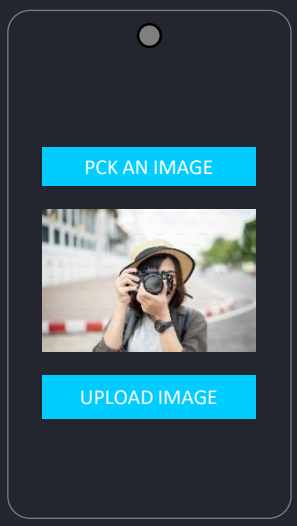


React Native Upload Image

1 `npx expo install expo-image-picker`

2 App.js

```
import React, { useState } from "react";
import { View, Button, Image, Text } from "react-native";
import * as ImagePicker from "expo-image-picker";

export default function UploadImageExample() {
  // State to store the selected image URI
  const [image, setImage] = useState(null);

  // Function to pick an image from the device gallery
  const pickImage = async () => {
    const result = await ImagePicker.launchImageLibraryAsync({
      mediaTypes: ImagePicker.MediaTypeOptions.Images, // Only images
      allowsEditing: true, // Allow basic editing
      aspect: [4, 3], // Image aspect ratio
      quality: 1, // Image quality (1 = max)
    });

     // If the user didn't cancel, set the image URI
      if (!result.canceled) {
     setImage(result.assets[0].uri);     }
  };

  // Function to upload the selected image to the server
   const uploadImage = async () => {
    if (!image) return; // Don't upload if there's no image selected

    const formData = new FormData();
    formData.append("file", {
      uri: image, // Image URI
      name: "photo.jpg", // File name
      type: "image/jpg", // File type
    });

    try {
      // Send the image to the API endpoint
      const response = await fetch("https://your-api-endpoint.com/upload", {
        method: "POST",
        body: formData,
        headers: {
          "Content-Type": "multipart/form-data",
        },
      });

      const result = await response.json();
      alert("Upload Success: " + JSON.stringify(result)); // Show success message
    } catch (error) {
      alert("Upload Failed: " + error.message); // Show error message
    }
  };

  return (
    <View style={{ flex: 1, justifyContent: "center", alignItems: "center" }}>
      <Button title="Pick an image" onPress={pickImage} />

      {image && (<Image source={{ uri: image }} style={{ width: 200, height: 200 }} />)}
     {image && <Button title="Upload Image" onPress={uploadImage} />}
    </View>
  );
}
```



1 App.js Example of : React.memo

```
import React from "react";
import { Text } from "react-native";

const Mycomponent = React.memo(({ value }) => {
  console.log("Rendered!");
  return <Text>{value}</Text>;
});

export default Mycomponent;
```

Wrap your functional components with React.memo to prevent re-renders if the props don’t change.

2 App.js Example of : useCallback

```
import React, { useCallback, useState } from "react";
import { Button, Text, View } from "react-native";

const App = () => {
    const [count, setCount] = useState(0); // State variable to track the count

  // Function to increment the count (wrapped in useCallback for optimization)
  const increment = useCallback(() => {
    setCount((prev) => prev + 1); // Increment count by 1
  }, []); // Dependencies array is empty, so the function reference won't change

  return (
    <View>
      <Button onPress={increment} title="Increment" />
      <Text>{count}</Text>
    </View>
  );
};

export default App;
```

---

useCallback: Memoizes functions so they don’t change on every render.useMemo: Memoizes expensive calculations.

3 App.js Example of : FlatList Optimally

```
<FlatList
   data={data} // Data array to render in the list
   keyExtractor={(item) => item.id.toString()} Unique key for each item (converted to string)
   getItemLayout={(data, index) => ({Optimize scrolling performance by specifying item layout
    length: ITEM_HEIGHT, // Fixed height of each item
    offset: ITEM_HEIGHT * index, // Distance from the top for the current item
    index, // Index of the current item
  })}
  // Render each item as a Text component displaying the item's name
  renderItem={({ item }) => <Text>{item.name}</Text>}
  windowSize={5} // Number of screens worth of items to keep in memory
/>;
```

FlatList is more efficient than ScrollView for rendering large datasets. Follow these tips for optimal usage:

1) Always provide a unique keyExtractor to minimize re-renders.jsxCopy code
2) If the height of each item is fixed, use getItemLayout for faster scrolling.
3) Use initialNumToRender and windowSizeControl how many items are rendered initially and offscreen

1 App.js Example of : Avoid Inline Functions

```
// Memoize the renderItem function to prevent unnecessary re-renders
const renderItem = useCallback(
  // Render each item in the list
  ({ item }) => <Text>{item.name}</Text>,
  [] // Dependencies (empty means it won't change)
);

// Render the list using FlatList
<FlatList
  data={data} // The array of data to display
  renderItem={renderItem} // The function to render each item
/>;


-----------------------------------------------------------------------------------------------
Inline functions can cause re-renders. Use useCallback for the renderItem function.jsxCopy
code
```

1 App.js Example of : Image

```
import { Image } from "react-native";

Image.prefetch("https://example.com/image.jpg");


-----------------------------------------------------------------------------------------------
Optimizing Images and Assets
-----------------------------------------------------------------------------------------------
Large images and unoptimized assets can slow down your app. Follow these tips:

a. Use react-native-fast-image (if not using Expo)For Expo users, prefer resizing images or
   caching strategies.

b. b. Resize ImagesCompress images to reduce file size without sacrificing quality. Tools
   like Squoosh can help.

c. Use Image.prefetchPreload images before rendering.jsxCopy code
```

1 App.js Example of : SVGs for Icons

```
import { Svg, Circle } from "react-native-svg";

const MySvg = () => (
  <Svg height="100" width="100">
    <Circle cx="50" cy="50" r="50" fill="blue" />
  </Svg>
);

-----------------------------------------------------------------------------------------------
npx expo install react-native-svg
-----------------------------------------------------------------------------------------------
Use SVGs for IconsInstead of using large PNG or JPG icons, use vector-based SVGs.Use react-
native-svg (available in Expo).
```

React-Native Expo Command List

1. npx create-expo-app@latest --template blank@latest

2. npx start -- --reset-cache (Reset the cache)

3. npx expo start

8. npx expo-doctor (for fix issue)

8. npm install -g expo-cli (upgrade expo cli)

1 app.json

```
Prepare Your App for Releasea.
Set Up App Metadata in app.
json (or app.config.js)In your Expo project, update the following in the app.json file:
-----------------------------------------------------------------------------------------

{
  "expo": {
    "name": "Your App Name",
    "slug": "your-app-slug",
    "version": "1.0.0",
    "orientation": "portrait",
    "icon": "./assets/icon.png",
    "splash": {
      "image": "./assets/splash.png",
      "resizeMode": "contain",
      "backgroundColor": "#ffffff"
    },
    "android": {
      "package": "com.yourcompany.yourappname",
      "versionCode": 1,
      "permissions": []
    },
    "ios": {
      "bundleIdentifier": "com.yourcompany.yourappname",
      "buildNumber": "1.0.0"
    }
  }
}
-----------------------------------------------------------------------------------------
android.package: Use a unique package name, like
com.yourcompany.yourappname.ios.bundleIdentifier: Use a unique identifier, like
com.yourcompany.yourappname.Add icons, splash screens, and other metadata as needed.
```

2 **npm install -g expo-cli** : Build Your App, Install Expo CLIEnsure you have the latest version of Expo CLI:

3 **expo login** : Login to Expo, Log in to your Expo account (or create one if you don't have it):

4 **expo build:android** : Run the Build Command, For Android APK/AAB:

5 **expo build:ios** : For iOS App:

```
Expo will ask whether to use an Expo-managed workflow or custom credentials. For simplicity,
let Expo manage the credentials.


d. Download the Build
Once the build is complete, Expo will give you a link to download the APK (Android) or IPA
(iOS).


3. Testing Your BuildAndroid:
Install the APK directly on your device to test it.iOS: Use TestFlight to test your app on
iOS devices. (You'll need to upload the IPA to App Store Connect first.)
```

1 Publish Apk on : Google Play Store

Create a Google Developer Account

Sign up at Google Play Console.
Pay the one-time registration fee (~$25).

Create a New App

In the Google Play Console, click “Create App” and provide your app details.

Upload the Android Build

Navigate to the "Production" track under "Release" > "Create New Release."
Upload your AAB file (preferred) or APK file.

Fill Out Store Listing

Add a description, app screenshots, category, contact details, and privacy policy.

Submit for Review

Once complete, click "Submit" to review and publish your app.

2 Publish Apk on : Apple Store

1.Enroll in the Apple Developer Program

•Sign up at Apple Developer.
•The fee is $99/year.

2.Create an App Record in App Store Connect

•Go to App Store Connect and create a new app record.

3.Upload the iOS Build

•Use **Transporter** (a macOS app) to upload your IPA file to App Store Connect.

4.Configure App Store Metadata

•Add app descriptions, screenshots, categories, and other details.

5.Submit for Review

•After setting everything up, submit the app for Apple’s review process. Once approved, it will go live.

1 expo eject

Ejecting from Expo
Ejecting refers to converting your managed Expo project into a bare React Native project. This gives you more flexibility but also requires additional setup and maintenance

---

When to Eject
You should consider ejecting only if:
1.You need native code:
•Integrating custom native modules not supported by Expo (e.g., some SDKs like Stripe or custom camera functionality).
2.You need advanced customizations:
•Changing the build process (e.g., custom Gradle settings in Android or modifying Info.plist in iOS).
3.You want more control over the app's lifecycle:
•You might need access to native features unavailable in Expo's managed workflow.
4.Dependency restrictions:
•If a third-party library doesn't work with Expo's managed workflow.
If you're not facing these issues, **staying with Expo** is easier and more efficient.

Steps for Ejecting
Ejecting from Expo converts your project into a React Native project with full native code access. Here's how you do it:
Step 1: Backup Your Project
Ejecting changes your project structure and files significantly. Make sure to back up your codebase.
Step 2: Run the Eject Command

Step 3: Choose Bare Workflow
You'll be prompted to select whether you want to:
1.Use the **Bare workflow**.
2.Continue using some of Expo's services (e.g., OTA updates, managed builds).
Step 4: Install Dependencies
Ejecting requires native build tools. You'll need to install:
•**Node.js** (already required by Expo).
•**Android Studio** (for Android development).
•**Xcode** (for iOS development, macOS only).
Expo will guide you through installing the necessary dependencies.
Step 5: Modify Your Build Files
Once ejected, you'll have access to:
•android/ folder for Android-specific code.
•ios/ folder for iOS-specific code.
Make changes here if required. For example:
•Add custom native modules.
•Edit Gradle files (Android) or Info.plist (iOS).

Step 6: Test Your App
Run your app on physical or virtual devices: Bash

```
npx react-native run-android
npx react-native run-ios
```

What Changes After Ejecting?
1.New Folder Structure:
•You'll see android/ and ios/ directories in your project.
•These folders contain native code for your app.
2.Expo-Specific Features:
•You lose access to some managed features (e.g., Expo Go).
•However, you can still use Expo libraries like expo-location or expo-camera.
3.Native Build Process:
•You'll now need to handle native builds yourself using **Android Studio** or **Xcode**.
4.Increased Responsibility:
•You're responsible for configuring and maintaining native code and dependencies.

When NOT to Eject
If you don't need custom native features or build configurations, avoid ejecting.
For most apps, Expo's managed workflow (especially with services like EAS Build) is sufficient.